

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Data Engineering and Analytics

**Scaling Laws of Equivariant Convolutional
Neural Networks**

Stefan Frisch

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Data Engineering and Analytics

**Scaling Laws of Equivariant Convolutional
Neural Networks**

**Skalierungsgesetze für äquivalente
Convolutional Neural Networks**

Author:	Stefan Frisch
Supervisor:	Florian Hözl
Advisor:	PD Dr. med. Georgios Kaissis
Submission Date:	22.12.2023

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 22.12.2023

Stefan Frisch

Acknowledgments

I am immensely grateful to a number of people whose support and guidance have been invaluable during the journey of completing my master's thesis.

First and foremost, I would like to express my deepest gratitude to my supervisor, Florian Hölzl, for his invaluable feedback and unwavering support throughout this process. His ability to challenge my ideas has not only helped me to sharpen my focus on the most important topics but also encouraged me to bring out the best in my academic pursuits. His mentorship has been a cornerstone of my development and success in this endeavor.

I am also indebted to the Lab for AI in Medicine at TU Munich for providing the necessary resources and an environment conducive to learning and innovation. The facilities and academic atmosphere at the lab have been instrumental in the completion of my research.

A special word of thanks goes to my family, whose unconditional love and support have been my backbone. Their endless encouragement and belief in my capabilities have been a source of strength and motivation, especially during the most challenging phases of my work.

I am fortunate to have friends like Bryson Pacheco, Tim Schekel, David Dunsen, Oleg Olychef, and Tim Maier, who have been more than just companions. Their unique perspectives, constant motivation, and unwavering friendship have played a significant role in keeping me focused and balanced. The camaraderie and support they have provided are beyond words and have greatly enriched my journey.

To all of you and everyone I forgot to mention, I am eternally grateful. Thank you for being part of this journey.

Abstract

Convolutional Neural Networks (CNNs) have revolutionized the field of computer vision, largely due to their inherent ability to exploit translational symmetries in images. Building on this success, there has been a growing effort to leverage rotations and reflections. Early studies on rotation and reflection Equivariant Convolutional Neural Networks (ECNNs) have demonstrated improved sample efficiency and better generalization than traditional CNNs. However, despite their potential advantages, ECNNs have yet to be widely adopted in practical applications. This limited adoption is partly due to the fact that current ECNNs frequently fail to surpass CNNs in performance, compounded by their unexplored design space and computational overhead. Our study aims to bridge this gap by reexamining and applying the foundational design principles of CNNs within the context of ECNNs.

We introduce a novel baseline architecture, Eq-NASNet, developed through a multiobjective neural architecture search. This search optimizes computational efficiency and performance, tailoring the architecture for practical applications. A key contribution of our research is the empirical demonstration of scaling laws for ECNNs. We systematically investigate how width, depth, and resolution scaling impact the performance and computational complexity of Eq-NASNet.

Our rigorous empirical evaluations highlight the superiority of Eq-NASNet over established models like EfficientNet and Vision Transformer (ViT) in challenging medical image classification scenarios. Notably, Eq-NASNet is more parameter efficient and matches the computational efficiency of current architectures like EfficientNet, while displaying superior performance. Furthermore, we demonstrate Eq-NASNet’s robustness and effectiveness in low-data regimes and against adversarial attacks, where it distinctly outperforms both EfficientNet and ViT. The outcomes of our study position Eq-NASNet as a superior alternative for challenging medical tasks, where its ability to exploit increased image symmetry offers significant advantages.

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
2. Theory	3
2.1. Group	3
2.2. Representation	5
2.3. Convolutional Neural Networks	6
2.4. Equivariant CNNs	7
2.5. Generic ECNN Architecture	10
3. Methods	12
3.1. Design Choices	12
3.1.1. Symmetry Group Design Choices	12
3.1.2. Block Content	15
3.1.3. Scaling of ECNNs	15
3.2. Manual Architecture Adaption	16
3.2.1. Equivariant Wide Residual Network	16
3.2.2. Group Experiments with Eq-WRN	17
3.2.3. Block Content Experiments with Eq-WRN	18
3.3. ECNN Architecture Search	19
3.3.1. Search Space	19
3.3.2. Select a Baseline Architecture	23
4. Results	25
4.1. Scaling with Manual Architecture Adaptions	25
4.1.1. Symmetry Group for Eq-WRN	25
4.1.2. Optimal Block Content for Eq-WRN	28
4.1.3. Scaling with Eq-WRN	29
4.2. ECNN Baseline Architecture	31
4.2.1. Neural Architecture Search	31

4.2.2. Select and Analyze Eq-NASNet	32
4.3. Scaling Laws for ECNNs	37
5. Comparison to Leading Image Classification Architectures	40
5.1. Medical Data	41
5.2. Efficiency	43
5.3. Low Data Regime	45
5.4. Adversarial Attacks	47
6. Conclusion	49
A. Appendix	51
A.1. Trainings Setup for Different Datasets	51
A.2. Hyperparameter Optimization	52
Abbreviations	54
List of Figures	55
List of Tables	57
Bibliography	59

1. Introduction

Images exhibit symmetries that can be exploited for neural network architecture design to improve sample efficiency and generalization. Convolutional Neural Networks (CNNs) have seen significant success by leveraging translational symmetries in images [Lec+98; KSH17]. In recent years, there has been a notable trend in enhancing the capabilities of CNNs to utilize rotation and reflection symmetries through group actions. This development has emerged as a powerful inductive bias. Incorporating this additional symmetry, often referred to as equivariance, within network layers enables the predictable transformation of features in response to transformations in the input space. Pioneering work in this field on rotation equivariance through group convolutions was done by Cohen and Welling [CW16a]. Later, they expanded their research to steerable CNNs [CW16b]. This concept has since been adapted for 3-dimensional scenarios [Wei+18] and continuous groups [CGW18; CGW20]. Weiler and Cesa further contributed to the field by addressing the kernel constraint for arbitrary representations using irreducible representations [WC19; CLW22] and facilitating the construction of Equivariant Convolutional Neural Networks (ECNNs) through the development of a specialized library.

For CNNs, the design choices and trade-offs in performance versus compute have been extensively explored. Studies such as Wide Residual Networks [ZK17] by Zagoruyko and Komodakis, highlight the importance of scaling depth and width in networks. Designing networks for compute-limited devices has pushed for more efficiency in CNNs [How+17]. Sandler et al. [San+19] improved efficiency even further by searching for a compute versus performance optimal architecture. In EfficientNet, the authors showed the potential to boost the networks' capabilities by simultaneously scaling depth, width, and resolution [TL20]. This exploration has resulted in highly optimized CNN architectures tailored for common tasks like natural image classification, setting the stage for further advancements through specialization.

Some early studies, such as the general $E(2)$ study [WC19], have begun to compare performance across different approaches to incorporate additional symmetries. Further, research has indicated that an equivariant version of MobileNet, designed for compute-limited environments, can improve performance on medical image data [Moh+20].

This was combined with quantization techniques to reduce computational demand with only marginal performance sacrifices. However, these preliminary studies have yet to offer a systematic, comprehensive exploration of the trade-offs and optimal design choices in the context of ECNNs. This is especially true if we consider that the huge interest in designing CNN architectures has been fundamental to their widespread success and good performance.

This work aims to extend the knowledge about ECNNs and their design decisions and trade-offs. The key contributions of this work are:

1. Introduction of a novel architecture family, Eq-NASNet, for ECNNs. This family originates from a multiobjective neural architecture search, optimizing both computational efficiency and performance.
2. Proposal and demonstration of scaling laws in depth, width, and resolution for ECNNs. The scaling of ECNNs enables increased performance with a greater computational budget.
3. Demonstration of Eq-NASNet’s superior performance over established architectures like EfficientNet and ViT on challenging medical datasets while it matches the computational efficiency. Furthermore, we show it matches or even outperforms existing pre-trained versions of these networks.
4. Eq-NASNet excels in low-data scenarios and demonstrates increased robustness against adversarial attacks, making it highly practical for real-world applications.

2. Theory

This chapter explores the theoretical framework underpinning discrete rotation and reflection ECNNs. The discussion begins with a foundational overview of group theory and representations, essential for understanding the mechanics of ECNNs. We then revisit the core mathematical principles of convolutions from CNNs, followed by an extension to group convolutions/cross-correlations. The chapter ends with a combination of these concepts by presenting a generic ECNN architecture.

2.1. Group

A **group** is a tuple of a non-empty set G and a binary operation $\cdot$ that acts on the elements in G .

$$\cdot : G \times G \rightarrow G, (a, b) \mapsto a \cdot b \quad (2.1)$$

satisfying the three group axioms:

- Associativity: $\forall a, b, c \in G : (a \cdot b) \cdot c = a \cdot (b \cdot c)$
- Identity element: $\exists e \in G$ s.t. $\forall a \in G : e \cdot a = a \cdot e = a$
- Inverse element: $\forall a \in G, \exists a^{-1} \in G : a \cdot a^{-1} = a^{-1} \cdot a = e$

In this work, we adopt the conventional notation ab to represent $a \cdot b$. A **subgroup** is a group with the same binary operation $\cdot$, but where H is a subset of G . We will only cover groups with a finite number of elements, called **finite groups**.

A central role in the following plays the **cyclic group** of order N . It covers the discrete rotations by angles that are multiples of $\frac{2\pi}{N}$:

$$C_N = \left\{ e^{ik\frac{2\pi}{N}} \mid 0 \leq k < N \right\} \quad (2.2)$$

The **group order** can generally be defined as the number of elements in the set G from the group definition.

The layers of neural networks that process images are, from a mathematical perspective, functions over a plane. To describe the symmetry of functions on planes, it is useful to define **group actions**. Given a group G and a set X , then a (left) group action of G on X is a function

$$G \times X \rightarrow X, (g, x) \mapsto g \cdot x \quad (2.3)$$

that satisfies the following two axioms:

- **Identity:** $\exists e \in G$ such that $e \cdot x = x$
- **Compatibility:** $\forall a, b \in G, \forall x \in X : a(bx) = (ab)x$

With these fundamentals in place, we are prepared to explore three specific group actions that capture all the essential symmetries for our discussion:

- **Translation:** Given the translation group $T = (\mathbb{R}^2, +)$, the action of the translation group on the Euclidean plane is defined as follows:

$$\forall t \in (\mathbb{R}^2, +), \forall (x, y) \in X = \mathbb{R}^2 : t \cdot (x, y) \mapsto t \cdot (x, y) = (x + t_1, y + t_2) \quad (2.4)$$

- **Rotation:** With the cyclic group C_N from Equation 2.2 the rotation action can be defined by:

$$\forall r_k \in C_N, \forall x \in X = \mathbb{R}^2 : r_k \cdot x \mapsto r_k \cdot x = R\left(\frac{2\pi}{N} \cdot k\right)x = R(\theta)x \quad (2.5)$$

where:

- $\theta \in \frac{2\pi}{N} \cdot k$ for $k \in \{1, 2, \dots, N\}$ is an angle
- $R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$ is the rotation matrix
- **Reflection:** The reflection group $F = (\{\pm 1\}, *)$ is characterized by the action f , which orthogonally reflects either along the x or y-axis.

$$\forall f \in F, \forall x \in X = \mathbb{R}^2 : f \cdot x \mapsto f \cdot x \quad (2.6)$$

To extend network equivariance beyond the translation group found in CNNs, it becomes necessary to combine groups. Mathematically, combining groups can be expressed by the **semi-direct product**, denoted by $G \rtimes_{\phi} H$, where ϕ is an action $G \times H \rightarrow G$. Let $(G, +)$ and $(H, *)$ be groups, then the Cartesian product of G and H ($G \times H$), forms a set. This set can be endowed with a binary operation, as defined below:

$$\cdot : (G \times H) \times (G \times H) \rightarrow (G \times H), (g_1, h_1) \cdot (g_2, h_2) = (g_1 + \phi(g_2, h_1), h_1 * h_2) \quad (2.7)$$

We obtain different groups depending on the map ϕ . For instance, combining the translation group $(\mathbb{R}^2, +)$ and the cyclic group C_N via the semi-direct product results in a subgroup of the special Euclidean group $SE(2)$, that only covers the discrete rotations up to frequency N . The group contains the elements (t_v, r_k) , where $t_v \in (\mathbb{R}^2, +)$ and $r_k \in C_N$. The corresponding group operation is expressed by:

$$\begin{aligned} (t_{v_1}, r_{k_1}) \cdot (t_{v_2}, r_{k_2}) &= (t_1 + \phi(r_{k_1}, t_{v_2}), r_{k_1} + r_{k_2}) \\ &= (t_{v_1} + R(\theta_1)t_{v_2}, r_{k_1} + r_{k_2}) \\ &= (t_{1+R(\theta_1)v_2}, r_{k_1+k_2}). \end{aligned} \tag{2.8}$$

Since the map $\phi : ((\mathbb{R}^2, +) \times C_N) \rightarrow (\mathbb{R}^2, +)$, $t_v, r_k \mapsto t_{R(\theta_1)v_2}$, produces $t_{R(\theta_1)v_2} \in (\mathbb{R}^2, +)$ the resulting structure is a well-defined group. We have used that the group operation on the translation and cyclic group is well-defined. The semi-direct product can be extended to combine three or more groups, thereby facilitating the extension with the reflection group. Therefore, it can combine all the transformation groups we need to build translation, rotation, and reflection equivariant networks.

2.2. Representation

In our previous discussion, we encountered a linear representation of the cyclic group $R(\theta)$. In this section, we will establish a formal definition of linear representations and their significance. In modern image processing, the images and features are represented as numerical matrices, which can be transformed through group actions in the form of linear representation. A **linear group representation** ρ is a map

$$\rho : G \rightarrow GL(V) \text{ such that } \rho(g_1 g_2) = \rho(g_1) \rho(g_2) \quad \forall g \in G \tag{2.9}$$

where $GL(V)$ is the general linear group of a vector space V . If the vector space $V = \mathbb{R}^n$, then $GL(V)$ is the set of $n \times n$ invertible matrices with the matrix multiplication as the group operation.

Three linear representations are particularly important in the following:

- **Trivial representation** $\rho : G \rightarrow GL(\mathbb{R})$ maps any group element to the identity $\forall g \in G : \rho(g) = 1$. This representation is used to transform images into group tensors since the channels (colors, grayscale) do not mix or change when the image is rotated.
- **Regular representation** $\rho : G \rightarrow GL(\mathbb{R}^{|G|})$ maps group elements to a unique permutation matrix. To make this more concrete, let's look at the cyclic group covering 4 rotations C_4 . The cyclic group has N many elements, so we end

up with four 4×4 permutation matrices, which are the identity matrix and the shifted identity matrices by one column at every step to the right. During the feature processing in the hidden layers of the ECNN, we rely on regular representations instead of other representations due to higher efficiency, as shown in [WC19].

- **Irreducible representation** do not contain nontrivial invariant subspaces. $R(\theta)$ is an example of irreducible representation (irreps). It can be proven that any real linear representation can be decomposed into irreps up to a change of basis. All calculations, such as solving the kernel constraint, are internally done on irreps in the ESCNN library [CLW22] that we build upon. We will not discuss irreps here further. Please see [WC19; CLW22] for the mathematical details.

Finally, we want to formally introduce the notion of equivariance. A linear map $f : X \rightarrow Y$ between two sets X, Y is said to be **equivariant** if and only if:

$$\forall x \in X, \forall g \in G : f(g \cdot x) = g \cdot f(x). \quad (2.10)$$

Intuitively, network layers are considered equivariant if the order of group transformation and feature processing does not matter. In other words, if we can apply the group transformation, such as image rotation by 90 degrees, before or after the feature processing, the result will be the same if the layer is equivariant.

2.3. Convolutional Neural Networks

CNNs leverage translational symmetries in images through convolutions. In practice, deep learning frameworks instead commonly implement cross-correlation. But since cross-correlation and convolution are equivalent up to kernel reflection and the fact that the kernels are learned, the distinction is inconsequential. We want to build our theory of ECNNs around cross-correlation as it intuitively shows the pattern matching in CNNs. **Cross-correlation** $k \star f$ between the two signals, kernel k and feature map (or input image) f can be defined as:

$$[k \star f](\Delta x) := \int_x k(x - \Delta x) f(x) dx \quad (2.11)$$

where Δx is the set of all translations. It can be proven that cross-correlation is equivariant to translations, meaning $\forall t \in T : [k \star t \cdot f] = t \cdot [k \star f]$. The translational equivariant bias in the convolutional layers increases weight sharing, which leads to higher parameter efficiency and better generalization in CNNs [Lec+98; KSH17].

2.4. Equivariant CNNs

To take advantage of symmetries in images beyond translations, we introduce the concept of group cross-correlation. Given the signals k, f over an input vector space V endowed with the group G , then a **group cross-correlation** is defined as:

$$[k \star_G f](g) := \int_{x \in V} k(g^{-1}x) f(x) dx \quad (2.12)$$

It can be proven that group cross-correlation is equivariant to group actions $\forall g \in G$: $[k \star_G g \cdot f] = g \cdot [k \star_G f]$ [WC19]. Since all groups g in this work are semi-direct product groups $g \in (\mathbb{R}^2, +) \rtimes_\phi H$, where H is either the cyclic group or the dihedral group, we can write g as (t_x, h) . Therefore, the Equation 2.12 transforms into:

$$[k \star_G f](y, h) = \sum_{h' \in H} \int_{x \in \mathbb{R}^2} k(h^{-1}(x - y), h^{-1}h') f(x, h') dx \quad (2.13)$$

Where we have used $g^{-1}g' = h^{-1}t_{-y}t_xh' = t_{h^{-1}(x-y)}h^{-1}h'$ with $g^{-1} := h^{-1}t_{-y}$ and $g' := t_xh'$. By expanding Equation 2.13 we arrive at [WC19]:

$$\underbrace{\begin{bmatrix} [k \star f]_e \\ [k \star f]_{h_1} \\ \vdots \\ [k \star f]_{h_{|H|-1}} \end{bmatrix}}_{k \star f: \mathbb{R}^2 \rightarrow \mathbb{R}^{|H|}} = \underbrace{\begin{bmatrix} k_e & k_{h_1} & \cdots & k_{h_{|H|-1}} \\ h_1 \cdot k_e & h_1 \cdot k_{h_1} & \cdots & h_1 \cdot k_{h_{|H|-1}} \\ \vdots & \vdots & \ddots & \vdots \\ h_{|H|-1} \cdot k_e & h_{|H|-1} \cdot k_{h_1} & \cdots & h_{|H|-1} \cdot k_{h_{|H|-1}} \end{bmatrix}}_{\mathbf{k}: \mathbb{R}^2 \rightarrow \mathbb{R}^{|H|} \times |H|} \star \underbrace{\begin{bmatrix} f_e \\ f_{h_1} \\ \vdots \\ f_{h_{|H|-1}} \end{bmatrix}}_{\mathbf{f}: \mathbb{R}^2 \rightarrow \mathbb{R}^{|H|}} \quad (2.14)$$

The number of rows of the kernel matrix of the group cross-correlation depends on the number of group elements of the input feature map. In other words, it depends on the representation of the data. For the first so-called lifting convolution, the input feature map encompasses the number of input image channels. Since the input image is made up of channels C and these channels do not transform or mix under rotations or reflections. Thus each input channel corresponds to one trivial representation. After lifting the input image to the group, the data is represented as a regular representation. Thus, the feature map vector $\mathbf{f}$ has group order $|H|$ many elements. The shape of kernel matrix $\mathbf{k}$ is $|C| \times |H|$ for the lifting convolution and $|H| \times |H|$ for all intermediate convolutions. Notably, the degree of freedom for variation in $\mathbf{k}$ is limited, specifically to the first row, as the remaining rows are transformed versions affected by the action h . The process of expanding the first row of $\mathbf{k}$ by the appropriate action h is called basis expansion. During the training phase, the learnable parameters in the first row are continuously updated, and a basis expansion is performed at every step

to obtain the complete kernel matrix. However, during evaluation, we can store the entire kernel matrix without incurring additional overhead compared to conventional CNNs. For an illustration of this process, see Figure 2.1.

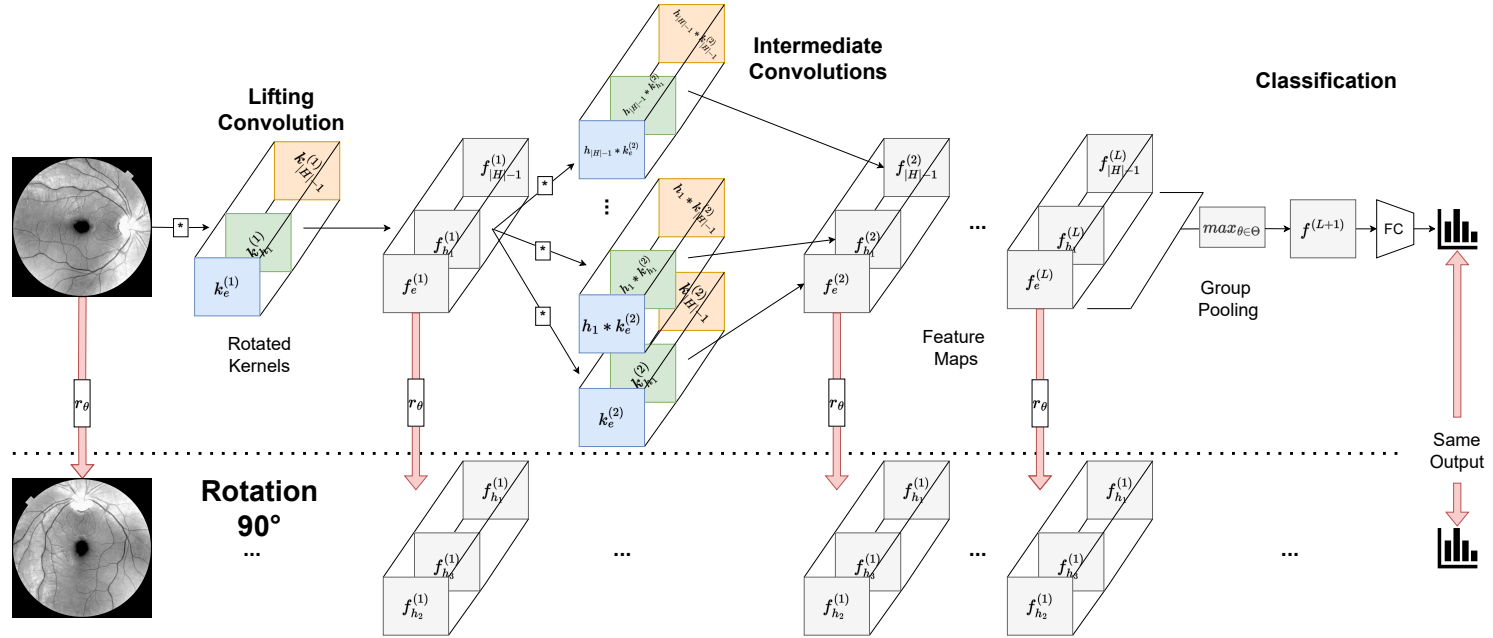


Figure 2.1.: Illustration of rotation equivariance processing with an ECNN from a theoretical perspective inspired by [WHS18] using the human eye [Com23a] as an example. $f_{h_j}^{(i)}$, $k_{h_j}^{(i)}$ represent the feature map $\mathbf{f}$ and kernel $\mathbf{k}$ in layer i endowed with the group action h_j . Filter kernels in the same color share weights since these are the same kernels but rotated by the group action h . The orientation of the symbols illustrates the rotation of the kernel.

2.5. Generic ECNN Architecture

In this section, we leave the mathematical details to the side and focus on developing a generic ECNN architecture, contrasting it with the familiar structure of traditional CNNs to underscore key differences and implications for architectural design.

Lifting The ECNN architecture diverges from CNNs at the first convolution. In ECNNs, the input image is first transformed into a group tensor with the trivial underlying representations. This group tensor is then lifted to the group with the lifting convolution. This process is similar to the initial convolution in CNNs, sharing design choices like kernel size and stride. However, ECNNs introduce an additional design element: the choice of the initial group. This group choice remains constant throughout the equivariant feature processing in ECNNs, adding a layer of freedom not found in CNNs.

Central The central structure of an ECNN mirrors that of a CNN, often comprising convolutional layers, batch normalizations, and activation functions. The fundamental difference lies in the use of group convolutions in ECNNs. These group convolutions ensure that the network’s equivariance to the chosen symmetries is maintained. The equivariance level of a network does not have to remain constant throughout the whole network. Depending on the symmetry level in the dataset, it is often useful to restrict the symmetry group in deeper layers of the network.

The number of feature maps of a group convolution is calculated differently than for normal convolution. In normal convolutional layers of CNNs, the output channel size is equal to the number of feature maps produced. However, for group convolutions, it is determined by the number of representations multiplied by their underlying group order. We introduce the notion of the **effective number of channels** or **effective channel size** for this multiplication, which allows us to extend the concept of width naturally to ECNNs.

Classification Before classifying an image using ECNNs, a crucial decision needs to be made regarding the orientation information in the task. This choice guides whether to maintain or pool equivariant features.

In tasks where the orientation of objects does not add information, such as classifying types of skin lesions in dermatology, pooling over the group is beneficial. Here, the focus is on detecting specific features irrespective of their orientation. Therefore, creating group invariant features through pooling operations like average or max pooling over the group feature maps of each representation is helpful.

On the other hand, for tasks where the orientation holds substantial value, such as in traffic analysis, where identifying the direction a vehicle is facing is key, it is essential to preserve the orientation features. In these scenarios, skipping the pooling step allows the network to effectively classify orientation-specific details, utilizing the full potential of ECNNs in capturing and processing orientation-dependent information. The final stage in ECNNs, involving fully connected layers and a softmax function, is generally similar to that in CNNs.

Designing an ECNN architecture, while closely connected to CNNs, demands some rethinking. The upcoming sections, especially Section 3.1, will explore these design choices in depth, comparing and contrasting them with those in CNNs. This comparison will not only clarify the unique considerations in ECNN design but also provide insights into how established CNN principles are adapted in ECNNs.

3. Methods

This chapter presents our methodology for investigating the design space of ECNNs. We focus on understanding their similarities and differences from CNNs and the scaling behavior. Therefore, we present an equivariant implementation of Wide Residual Networks (WRNs) [ZK17] and our approach to finding a new compute versus performance optimal ECNN baseline architecture.

3.1. Design Choices

ECNNs present various design choices and trade-offs. The considerations involves the design decisions associated with CNNs, such as depth, number of feature maps, kernel size, and design choices specifically linked to the symmetry group. Our discussion begins by examining the symmetry group choices, emphasizing how these intersect with standard network design elements and influence computational costs. We then transition to exploring conventional CNN design elements and how they apply or adapt in the context of ECNNs. We focus especially on the scaling behavior and its implications for performance and efficiency.

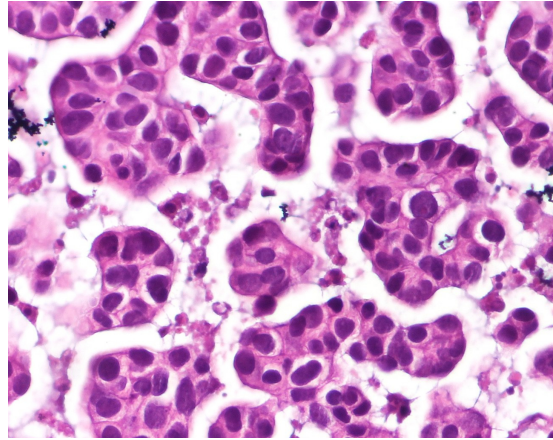
3.1.1. Symmetry Group Design Choices

Symmetries in Images Images exhibit diverse symmetries at different scales. In particular small image areas (few pixels) typically exhibit patterns like edges in arbitrary orientations. But even at slightly larger scales, rotational and reflectional symmetries are common. Take, for instance, the sunflowers depicted in Figure 3.1a. The sunflower heads display an almost perfect rotational, reflectional, and translational symmetry. However, the rotational symmetry is lost at the larger scale since there is a clear up-and-down orientation as in most natural images. Full rotational symmetry at the global scale is rare, with some for the field of ECNNs particularly interesting exceptions like satellite or medical images. For example, the histopathology image in Figure 3.1b is approximately equivariant to rotations even at the highest level.

There are other kinds of symmetries, like symmetry of scale. For example, the small sunflower heads in the back of the image are the same object as the larger heads in the front. However, we did not investigate scale symmetries [WW19] or continuous



(a) Sunflower field [Com23b]



(b) Histopathology image [Com23c]

Figure 3.1.: Images can exhibit symmetries at different scales. Natural images like the sunflower field show rotational symmetries primarily at the local level, like in the sunflower heads. The rotational symmetry is lost at the larger scale due to a clear up-and-down orientation. For many medical images, like the histopathology image, the orientation should not impact the classification.

groups and quotient spaces [WC19] in our work since they are computationally more expensive and provide only modest performance improvements.

Design Choices for Group Convolutions In the lower levels of a CNN, filters such as edge detectors are often repeated in various orientations. ECNNs, through rotational and reflectional equivariance, encode these orientations inherently, reducing the number of unique filters that need to be learned. At deeper levels of CNNs, filter repetition across several orientations is less common but can presumably, up to a point, aid generalization in ECNNs.

However, equivariance to a large symmetry group is computationally costly and can also introduce interpolation artifacts. Images are commonly represented as values on a discrete pixel grid. On a discrete grid, only the kernel rotations 1, 2, and 4 can be realized without interpolation. For other rotations, interpolation becomes necessary, potentially introducing artifacts. To minimize these artifacts, larger uneven filter sizes, such as 5 or 7, have been suggested [WC19]. But larger filters increase the number of parameters and, by that the computational complexity.

In summary, while increased equivariance offers benefits, it also incurs additional computational costs and potential issues with interpolation artifacts. Intuitively, to optimally leverage the image symmetries, each layer’s symmetry group should ap-

proximately match the symmetry present at its receptive field. We investigate how to cost-effectively structure the symmetry group throughout the network in detail in section 3.2.2.

Computational Costs All other things equal, incorporating equivariance into a network’s architecture results in a computational overhead, where the level of equivariance is proportional to the computational costs. Consider a group convolution with a fixed number of regular representations with the underlying symmetry group C_N or D_N . For a fixed number of representations, the group order is directly proportional to the number of feature maps and, therefore, the computational costs.

To manage these costs, the number of representations is typically adjusted based on the group order. To keep the number of feature maps constant and computational costs similar, we can divide the feature maps by the group order to arrive at the number of representations. Since group convolutions are more parameter efficient than normal convolutions, it is common [WC19] to multiply by the square root of the group order to keep the number of parameters comparable to normal convolutions. The two adjustment methods are described by the following Equation:

$$\text{Number of representations} = \begin{cases} \frac{f}{N}, & \text{FLOPs constant} \\ \frac{f}{N} \cdot \sqrt{N}, & \text{Parameters constant} \end{cases} \quad (3.1)$$

Where N is the group order and f is the number of feature maps.

A critical trade-off emerges between feature map diversity and predictable equivariant behavior to a large number of symmetries. Normal convolutions have complete feature map diversity but provide no equivariance guarantees. The other extreme is not used in practice but helps understanding. A group convolution with a single representation with a large underlying symmetry group but obviously very limited pattern recognition capabilities. In practice, ECNNs often accept slightly higher computational costs for more feature maps. With a higher number of feature maps, it is possible to achieve pattern-matching capabilities similar to CNNs and also keep the equivariance property, but at higher costs. Another driver of computational costs are larger kernels to mitigate interpolation artifacts. Thus, a crucial aspect of our research has been to balance the performance benefits derived from increased equivariance against the resulting computational overhead.

3.1.2. Block Content

The introduction of residual blocks, as first presented in [He+15], has been instrumental in the development of modern convolutional networks. To take optimal advantage of the idea of residual connections, the content inside each block is vital. In CNNs, inside each block, there are usually a number of convolutions intermixed with activation functions and batch normalizations. Zagoruyko and Komodak [ZK17] showed that a kernel layout of two convolutions, each with kernel size 3 inside a residual block, denoted with 3×3 , yields optimal performance for WRN. Building upon this, Tan et al. [Tan+19] explored through Neural Architecture Search (NAS) various convolution types, kernel sizes, and skip operations to further optimize block content.

We adopt this sequential approach from the literature for finding the optimal block content for ECNNs. Initially, we use manual architecture adaptations with an equivariant implementation of WRN to identify an effective block composition. That is followed by a NAS to systematically improve the architecture.

3.1.3. Scaling of ECNNs

The first major step towards improving the performance of CNNs has been the introduction of skip connections [He+15]. Skip connections mitigate the vanishing gradient problem, allowing for deeper CNNs with improved performance. Since the beautiful work of Zagoruyko and Komodak [ZK17], the limitations of depth scaling in CNN are now well understood. They suggested that scaling networks only in depth is inefficient, and the width should be scaled simultaneously. Higher-resolution images have more features that can be used to enhance performance even further [Sze+15]. Tan and Le [TL20] demonstrated a CNNs diminishing return in performance for scaling up single dimensions and highlighted the existence of an optimal balance in scaling depth, width, and resolution.

The scaling behavior of ECNNs has to our best knowledge, not been studied previously. The close connection to CNNs suggests that scaling in width, depth, and resolution could improve performance, but the higher computational demands might lead to different behavior. An important question arises: do scaling laws, like those in EfficientNet, apply to ECNNs, and if so, does the behavior differ in detail?

To understand the scaling behavior of equivariant architectures, we initially increase the depth and width of an equivariant implementation of WRN. This step is crucial to evaluate the feasibility of scaling in ECNNs and the unique challenges. Given the

computational overhead, scaling up an equivariant implementation of a specific CNN architecture might be infeasible. Therefore, we additionally aim to establish a more efficient baseline architecture for ECNNs, which is better suited for scaling.

3.2. Manual Architecture Adaption

We implement an equivariant version of the Wide Residual Networks architecture by Zagoruyko and Komodakis [ZK17] since it is a commonly used baseline architecture for CNNs and it has been used by other researchers in the ECNN field [WC19]. Before we can scale up this Equivariant Wide Residual Network (Eq-WRN), we perform some preliminary experiments to find the optimal group structure and block content.

3.2.1. Equivariant Wide Residual Network

The Eq-WRN architecture is largely a one-to-one translation of CNN to ECNN, with some noteworthy details. The lifting and classification parts are the same as for generic ECNNs described in chapter 2.5 and are not described in detail. The input is lifted to the initial group g_1 , followed by three layers of equivariant residual layers, possibly intermixed with group restrictions. After the third equivariant residual layer, group-max-pooling is applied. The remaining height and width of the feature map are then pooled by average pooling.

To maintain a comparable number of parameters to the CNN original, we employ the original channel size layout $[16, 16 * k, 32 * k, 64 * k]$, where k is the widen factor and apply Equation 3.1. We modify the shorthand for describing Eq-WRN from Zagoruyko and Komodakis [ZK17] as follows: Eq-WRN- n - k - $G(1-2-3)$, where depth $n = (N * 6) + 4$ and $G(1-2-3)$ indicates the group and number of rotations before the respective restrictions.

The equivariant residual layer is a one-to-one equivariant implementation of the original residual layer, with a minor modification in the skip connections. In the original design, the skip connection is consistently a residual connection. However, when the group is restricted, the residual connection cannot be applied, as the representations would be incompatible. In these instances, we apply a 1×1 equivariant convolution, which guarantees compatibility. The specifics of this architecture are summarized in Table 3.1.

Name	Output Size	Block Type
Lifting Conv	32×32	$[3 \times 3, 16, g_1]$
Layer1	32×32	$\begin{bmatrix} 3 \times 3, 16 \cdot k, g_1 \\ 3 \times 3, 16 \cdot k, g_1 \end{bmatrix} \times N$
Restriction1		$g_1 \rightarrow g_2$
Layer2	16×16	$\begin{bmatrix} 3 \times 3, 32 \cdot k, g_2 \\ 3 \times 3, 32 \cdot k, g_2 \end{bmatrix} \times N$
Restriction2		$g_2 \rightarrow g_3$
Layer3	8×8	$\begin{bmatrix} 3 \times 3, 64 \cdot k, g_3 \\ 3 \times 3, 64 \cdot k, g_3 \end{bmatrix} \times N$
Group-Max-Pool		$g_3 \rightarrow 1$
Average-Pool	1×1	$[8 \times 8]$

Table 3.1.: Our implementation of an Eq-WRN. The notation for the block $[3 \times 3, 16 \cdot k, g_1]$ in a Layer1 is as follows: 3×3 indicates two equivariant steerable convolutions separated by an activation function and a batch normalization, each with a kernel size of 3. $16 \cdot k$ marks the effective channel size. g_1 represents the group of the regular representations. N is calculated based on the depth.

3.2.2. Group Experiments with Eq-WRN

In exploring design choices related to rotation and reflection symmetry groups, our initial focus is to determine the optimal number of discrete rotations for the initial group g_1 . We incrementally increase g_1 from C_2 to C_{12} , assessing the impact on performance and Floating Point Operations (FLOPs) per training image.

The symmetry level at the most local level is probably similar across many different datasets. To further explore the impact of symmetry in deeper layers of the network, we look at three very diverse datasets: CIFAR10 [Kri09], MNIST rot [Lar+07], Galaxy10 DECals [LB18]. CIFAR10 involves the 10-class classification of small natural images (32×32 resolution). The task for MNIST rot adds a twist to the standard

MNIST handwritten digit classification problem by applying random rotations to all the images. For Galaxy10 DECals, 10 different galaxies have to be classified, which presents natural instances of rotation and reflection symmetries. These three datasets exhibit a diverse set of symmetries at different scales. Intuitively, the group of a layer should be determined by the symmetry level at the current receptive field. We conduct an experiment where we set the initial group g_1 to C_8 and systematically vary the subsequent groups ($g_{2,3}$) through group restrictions. This allows us to evaluate the influence of a perfect group choice versus a suboptimal group choice at different layers for a diverse set of images.

In addition to rotational symmetry, we also consider the role of reflectional symmetry. By introducing the reflectional bias into our models, we aim to assess the potential performance enhancements against the increased computational costs.

Given the potential need to increase kernel sizes for larger numbers of discrete rotations to mitigate interpolation artifacts, our last experiment examines the relationship between kernel size (specifically for rotation angles $\theta \leq \frac{\pi}{2}$) and the reduction of these artifacts. We conduct a dual-phase study: first, we increase the kernel size from 3 to 5 without adjusting for the parameter count increase, and second, we reduce the network’s width to maintain a constant number of parameters. This approach allows us to dissect performance improvements attributable to increased network capacity and the reduction of interpolation artifacts. The training setup is consistent for all our experiments, as detailed in section A.1.

3.2.3. Block Content Experiments with Eq-WRN

Building on the work of Zagoruyko and Komodakis [ZK17], we replicate their experiments on optimal residual block design. These experiments involve the kernel layout and the number of convolutions per block. We experiment with various kernel layouts including 3×3 , 1×3 , 3×1 , $3 \times 1 \times 3$, $3 \times 1 \times 1$, and $1 \times 3 \times 1$. Despite the potential influence of the number of discrete rotations on kernel size due to interpolation artifacts, we stick to the original kernel sizes from the Wide Residual Network study, treating interpolation artifacts as a separate issue related to symmetry groups. For a fair comparison, we maintain an approximately constant parameter count for both experiments by appropriately scaling either width or depth in line with the original study.

3.3. ECNN Architecture Search

The trade-off between performance and computational costs is a common theme in the design of ECNNs. Whether it is during the scaling of the network in depth and width, increasing the number of rotations, or the addition of reflectional symmetry. To our best knowledge, there are no studies available that focus on finding a performance versus compute optimal architecture specifically for ECNNs. ECNNs often require more computational resources during training, which makes a direct equivariant implementation of many well-known efficient CNN architectures prohibitively expensive to train. To overcome this problem, we use a multiobjective NAS to ensure that our new baseline architecture is both performant and low in resource consumption. We optimize for (weighted) validation accuracy and FLOPs with the Bayesian optimization framework BoTorch [Bal+20] and the Ax¹ API. FLOPs serve as a commonly used proxy for computational resource consumption [Zop+18; Rea+19] and are preferable to the number of parameters in our case.

3.3.1. Search Space

To efficiently explore the design space of ECNNs, we employ a similar factorized search space as MnasNet [Tan+19]. A factorized search space permits layer diversity but prevents the search space from exploding in size. Many NAS approaches before MnasNet only searched for a few complex cells and stacked these [ZL17; Liu+18], which does not permit layer diversity. In theory, layer diversity could be achieved by an individual search for all the content. However, that makes the search space too large for efficient exploration. Our search architecture consists of 4 main stages. The first part is a lifting convolution with 16 initial effective channels, which can be increased by the Increase Factor C_i . The initial effective channels must be divided by the group to determine the number of representations. Next, there are three blocks, each consisting of a possible restriction and a variable number of layers, as depicted in Figure 3.2. Notice we switch the terminology of layer and block compared to Eq-WRN to stay consistent with the terminology used in the literature. The content of the layers is searched for on a block level. The optimization engine can additionally add a 5th part, a head convolution with a possible restriction. The final classification part is the same as our equivariant implementation of WRNs and is not part of the optimization. All convolutions except the lifting convolution are preceded by a batch normalization and a Swish activation function [HG23]. Our architecture consists of fewer blocks than MnasNet or EfficientNet, as the number of FLOPs rises steeper with increased depth for ECNNs than for CNNs. Details on the search space are given by Table 3.2.

¹<https://github.com/facebook/Ax>

3. Methods

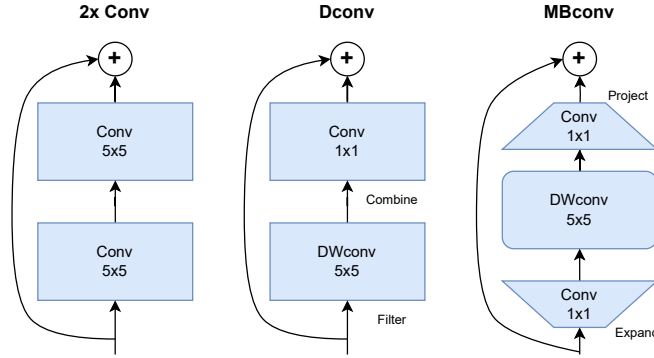
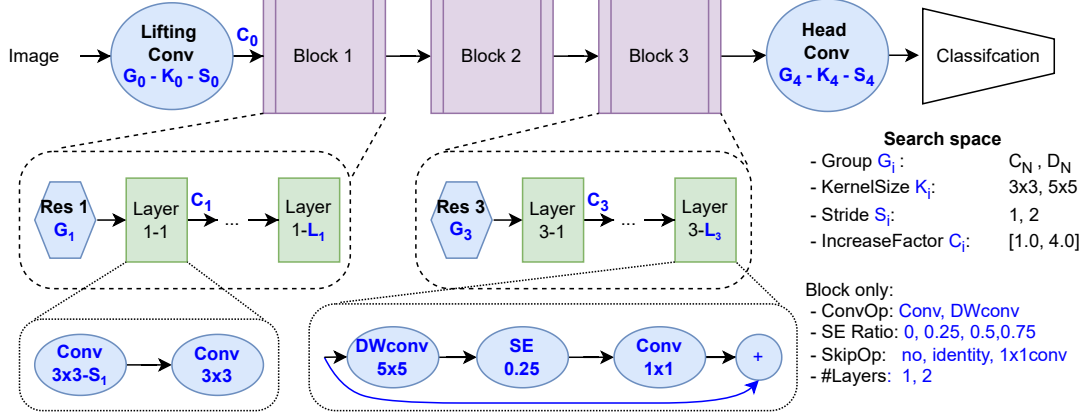


Figure 3.2.: Illustration of our factorized search space inspired by [Tan+19]. The network layers are organized into distinct blocks, each with specific input resolutions, groups, and effective channel sizes. A block consists of a possible group restriction (Res) and a variable number of layers. All layers of a block are the same except the first. The first convolution in the layer implements the stride and increase (C_i) of the effective channel size for the block. The figure illustrates the adaptability of our architecture skeleton, emphasizing the search for operations and connections within a single layer and the repetition across the L layers.

3. Methods

	Symbol	Options	Block	Lifting/Head
Group	G_i	C_N or D_N with $N \in \{1, 2, 4, 8, 16\}$	✓	✓
Kernel Size	K_i	3x3, 5x5	✓	✓
Increase Factor	C_i	[1.0, 4.0]	✓	✓
Stride	S_i	1, 2	✓	✓
Equivariant Convolution Operation	F_i	2x Conv, Dconv, MBconv	✓	
Squeeze-and-excitation ratio [Hu+19]	SE_i	0, 0.25, 0.5, 0.75	✓	
Skip Connection	SO_i	no skip, identity, 1x1 conv	✓	
Layers per Block	L_i	1, 2	✓	

Table 3.2.: Details on the options available during the NAS.

Group We restrict the Group G_i to cyclic or dihedral groups covering rotations 1, 2, 4, 8, and 16 since the number of rotations can only be reduced by a multiple of two. To cover all possible rotations would entail introducing a hierarchical search space, which is possible in Ax. Still, we refrain from doing so since the underlying optimization engine is unaware of the hierarchical search space and the complexity of encoding that quickly becomes unmanageable.

Equivariant Convolution Operation Depthwise separable convolutions (Dconv) [How+17] and mobile inverted bottlenecks (MBconv) [San+19] gained significant attention in the design of efficient CNN architectures:

- **Depthwise Separable Convolutions (Dconv)** reduce computational costs by decoupling the filtering and combining processes from standard convolutions. The initial depthwise convolution (DWConv) handles spatial filtering independently for each input channel. The following 1x1 pointwise convolution combines the

information from the separate channels. This approach significantly reduces the number of multiplications compared to standard convolutions by applying the larger kernel size only to a single channel (single representation in the equivariant implementation).

- **Mobile Inverted Bottlenecks (MBconv)** extends the idea of Dconv by incorporating an expansion phase followed by depthwise convolution and then a projection phase. Initially, the input is expanded by a 1×1 convolution to a higher-dimensional space, allowing the subsequent depthwise convolution to operate on a richer feature space. The final step involves projecting the features back to a lower-dimensional space, ensuring computational efficiency. While reflecting the bottleneck concept from [He+15], MBconv inverts this design, as it predominantly operates in the lower-dimensional spaces, expanding to higher dimensions only during the critical filtering phase.

Given the high computational costs typically associated with ECNNs, the integration of Dconv and MBconv into our search space can be valuable. Disregarding the Squeeze-and-Excitation ratio for a moment, the convolution operations inside each layer are MBconv, Dconv, or two regular group convolutions indicated with (Conv) as illustrated in Figure 3.2b. This approach ensures that our ECNNs can leverage state-of-the-art developments in CNN design, adapting these innovations to the unique demands and characteristics of equivariant networks.

Search Space Restriction To keep the search space small and use our available resources most effectively, unlike MnasNet, we do not search for the optimal expansion factor and dropout rate for each block. Instead, we search globally in (2, 4, 6) and (0.0, 0.1, 0.2, 0.3, 0.4, 0.5), respectively. We additionally impose the following restrictions on the search space:

- Stride: $\sum_{i=0}^4 S_i \geq 6$
- Increase Factor: $\sum_{i=0}^4 \lambda_i C_i \leq 6$
where λ_i are five points linearly spaced in the interval [1.0, 0.1]
- Group: the group order is none increasing: $\forall i, G_i \geq G_{i+1}$

where $i \in [0, 1, \dots, 4]$ indicates the block or $i = 0, 4$ the lifting convolution and head convolution, respectively. The stride and increase factor restrictions are necessary to prevent the optimization engine from suggesting too large architectures. Note in the increase factor inequality, we decrease the impact of C_i in deeper layers of the network, with λ_i to reflect the impact on the size and computational complexity of the network.

Dataset	Model Building Time [sec]	FLOPs [10^9]
MNIST rot	480	1.5
CIFAR10	600	2.3
Galaxy10 DECals	600	62.5
ISIC2019	600	62.5

Table 3.3.: Heuristic dataset-dependent upper bounds for stopping trials during the NAS.

Additionally, we impose dataset-dependent upper bounds on the model building time and the FLOPs to avoid overly long training times. The bounds in Table 3.3 were heuristically determined based on previous experiments. The information on the infeasible architecture is fed back into the optimization model to avoid similar suggestions.

3.3.2. Select a Baseline Architecture

Selecting an architecture from a range of potential candidates in this multiobjective optimization involves careful consideration of performance and computational costs. We present a four-step approach to arrive at the optimal baseline ECNN architecture:

1. **Selection on Pareto Frontier:** We focus on architectures on the Pareto frontier, which represents the optimal balance between performance and computational efficiency. In other words, for those architectures, no improvement can be made in one objective without compromising the other (see Figure 3.3). Depending on our computational constraints and performance goals, some architectures can be excluded at this stage.
2. **Comparative Analysis:** The goal is to identify common architectural features among the Pareto-optimal architectures across different datasets. This analysis helps in pinpointing a universally applicable architecture. Insights from the optimization models regarding the significance of specific architectural elements are also valuable in this step. Feature importance and marginal effect plots are accessible through the Ax API. The confidence in the model prediction can be assessed through cross-validation plots.
3. **Cross-dataset Evaluation:** After identifying a promising candidate, we assess its performance on various datasets. This step is crucial for evaluating the generalizability of the architecture. Its important to note that minor adjustments might be

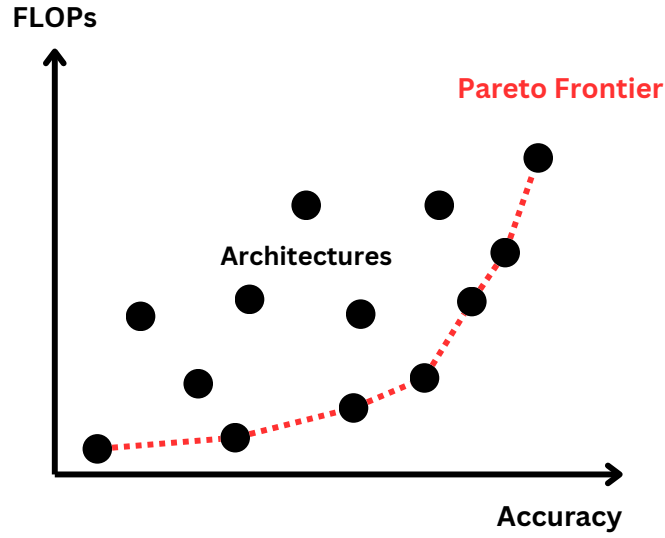


Figure 3.3.: Illustration of Pareto frontier for multiobjective optimization in performance and computational costs. The goal is to minimize FLOPs and maximize accuracy.

necessary to accommodate different image characteristics, such as size or symmetry levels. Depending on the performance, it might be necessary to return to step 2 and consider which architectural elements are responsible for the poor performance.

4. **Ablation Study:** In the final phase, we conduct an ablation study to analyze the critical components of the architecture. This study is aimed at understanding which features are essential for performance and which are more flexible. Any changes, especially those affecting computational demands, have to be compensated by width adjustment to guarantee a fair comparison.

This structured approach ensures a thorough and balanced evaluation of the generated architectures. It should simplify the discovery of an architecture that performs well, is computationally efficient, and is adaptable for various datasets and applications.

4. Results

This chapter gives some empirically grounded design suggestions for ECNNs. Guided by the results of the preliminary experiments with an equivariant implementation of WRN, we search for a compute versus performance optimal ECNN architecture. Additionally, we explore scaling ECNNs to increase performance if more computational resources become available.

4.1. Scaling with Manual Architecture Adaptions

To study the scaling behavior of Eq-WRN, we initially experiment with the symmetry group and the content (convolutions) inside each block. This approach ensures a reasonable starting architecture for our scaling experiments.

4.1.1. Symmetry Group for Eq-WRN

In this section, we study the impact of the symmetry group on Eq-WRN. Initially, we check what is the optimal number of discrete rotations for the initial group g_1 . Next, we look at the symmetry group for the deeper levels of the network. Given that different datasets have varying degrees of symmetry, we review this on three different. Then we examine the effect of adding reflectional symmetry to the group on network performance and computational resource consumption. Finally, we review how interpolation artifacts behave when we increase the kernel size. All the experiments in this section were performed with Eq-WRN-16-4 on CIFAR10 unless otherwise noted. The optimal restriction logic for CIFAR10 of halving the number of rotations (g_1) for g_2 and then restricting to the invariant in g_3 was determined in a prior experiment. We represent the restriction logic like this $C(X-X/2-1)$, where C indicates the cyclic group and the numbers are the rotations of the group before the respective restrictions.

Our studies suggest that increasing the number of rotations in the initial group improves performance up to 8 rotations, as depicted in Figure 4.1. However, this enhancement entails a proportional increase in FLOPs. Our findings are in line with Weiler and Cesa [WC19].

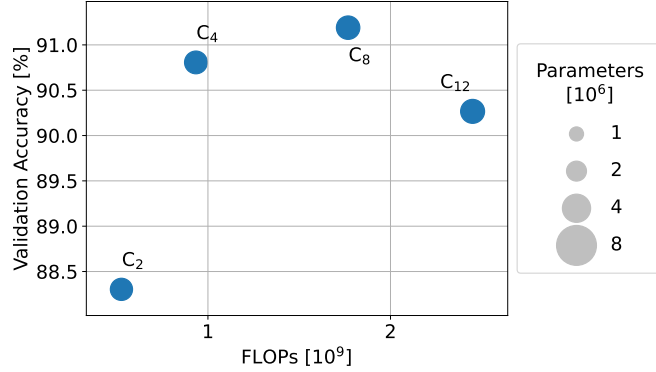


Figure 4.1.: Varying initial rotation levels for Eq-WRN-16-4-C(X-X/2-1) on CIFAR10.

Table 4.1 illustrates that the ideal level of restriction varies across different datasets, reflecting the level of symmetry at different scales. Datasets like MNIST rot, which are fully rotation-equivariant, benefit from no restriction, whereas datasets with natural rotation symmetry like Galaxy10 DECals favor partial restriction in deeper layers. In contrast, for natural images like in the CIFAR10 dataset, stricter restrictions and an earlier transition to invariance seem more effective. These observations follow the intuition that the symmetry present in the receptive field of a layer should determine the layer’s symmetry group. Although this seems complex, in practice, it is often straightforward. Fully rotation-equivariant datasets like MNIST rot are rare, and ECNNs are typically employed in scenarios where at least some level of equivariance, like in medical imaging or satellite data, is expected. A practical approach might be to start with a higher number of rotations (≈ 8) in the lifting convolution and progressively restrict them in deeper network layers.

The comparison between cyclic and dihedral groups, as presented in Table 4.2, reveals a modest performance increase of about 1% for the dihedral group, but at nearly double the computational cost. This recurring theme of balancing compute resources against performance is critical, leading us to the pivotal question: How can we design an ECNN that maximizes performance within a given computational budget? This question is addressed in section 4.2.

Further, we observe that larger kernel sizes improve performance, as shown in Table 4.3. Yet, contrary to previous assumptions [WC19], this improvement is not primarily due to reduced interpolation artifacts. When we adjust for parameter count by reducing the width, as in Table 4.4, the performance gains were negated, suggesting the performance gains predominantly stem from increased network capacity. There might be other phenomena at work, such as a reduction of the width below a certain thresh-

4. Results

Dataset	Restriction	Validation Accuracy [%]	FLOPs [10 ⁹]	Parameters [10 ⁶]
CIFAR10	C8C1C1	90.48	1.3	2.6
	C8C4C1	91.25	1.8	2.8
	C8C4C2	91.05	2.2	2.5
	C8C8C8	90.45	3.2	2.5
MNIST rot	C8C1C1	94.75	1.0	2.6
	C8C4C1	97.9	1.4	2.8
	C8C4C2	98.5	1.7	2.5
	C8C8C8	98.85	2.5	2.5
Galaxy10 DECals	C8C1C1	75.58	30.6	2.5
	C8C4C1	79.13	40.2	2.8
	C8C4C2	82.35	48.9	2.5
	C8C8C8	81.05	72.6	2.5

Table 4.1.: Varying restriction levels factors for Eq-WRN-16-4-X on different datasets. FLOPs can not be compared across datasets as they have different image sizes.

Model	Cyclic Group			Dihedral Group		
	Validation Accuracy [%]	FLOPs [10 ⁹]	Parameters [10 ⁶]	Validation Accuracy [%]	FLOPs [10 ⁹]	Parameters [10 ⁶]
16-4	91.25	1.8	2.8	92.21	3.3	3.0
22-6	92.78	6.0	9.3	93.10	11.6	9.8

Table 4.2.: Difference between the cyclic and dihedral group for different Eq-WRN sizes. The restriction logic remains the same.

old, which could hurt performance disproportionately. Since it is difficult to pinpoint all the potentially underlying reasons, we implicitly study this question again with NAS over the kernel size of the convolutions.

Initial Rotations	Kernel Size = 3×3			Kernel Size = 5×5		
	Validation Accuracy [%]	FLOPs [10^9]	Parameters [10^6]	Validation Accuracy [%]	FLOPs [10^9]	Parameters [10^6]
2	88.57	0.5	2.5	90.01	1.4	6.4
4	90.52	0.9	2.6	90.39	2.6	6.7
6	89.94	1.3	2.7	90.46	3.6	6.9
8	91.25	1.8	2.8	91.65	621	7.2
10	91.60	2.2	2.7	92.51	4.9	7.1
12	90.56	2.5	2.9	91.62	6.7	7.4

Table 4.3.: Relationship between kernel size and initial rotation level with **different parameter counts** for Eq-WRN-16-4-C(X-X/2-1) with on CIFAR10.

Initial Rotations	w = 4, Kernel Size = $\times 3$			w = 2.5, Kernel Size = 5×5		
	Validation Accuracy [%]	FLOPs [10^9]	Parameters [10^6]	Validation Accuracy [%]	FLOPs [10^9]	Parameters [10^6]
2	88.57	0.5	2.5	88.56	0.6	2.5
4	90.52	0.9	2.6	90.67	1.0	2.6
6	89.94	1.3	2.7	89.66	1.4	2.7
8	91.25	1.8	2.8	90.39	1.9	2.8
10	91.6	2.1	2.7	90.97	2.4	2.9
12	90.56	2.5	2.9	90.61	3.0	3.0

Table 4.4.: Relationship between kernel size and initial rotation level with **approximately similar parameter count** through width scaling for Eq-WRN-16-4-C(X-X/2-1) with on CIFAR10. When we increased the kernel size from 3×3 to 5×5 , we decreased the width from 4 to 2.5.

4.1.2. Optimal Block Content for Eq-WRN

Given the importance of the optimal kernel layout and number of convolutions per block in WRNs, we study these again for the equivariant implementation. We redo

Kernel Layout	Widen Factor	Validation Accuracy [%]	FLOPs [10 ⁹]	Parameters [10 ⁶]
3×3	4	91.25	1.8	2.8
1×3	5	89.70	1.8	2.5
3×1	5	89.10	1.3	2.4
$3 \times 1 \times 3$	4	88.61	1.9	2.9
$1 \times 3 \times 1$	5.2	88.17	2.1	3.0
$3 \times 1 \times 1$	5	88.52	1.5	2.7

Table 4.5.: Varying kernel layouts for Eq-WRN-16-X-C(8-4-1) on CIFAR10 while approximately controlling for parameter count through increasing the number of initial channels. Kernel layout 3×3 works best for both CNNs and ECNNs.

Deepening Factor	Depth	Validation Accuracy [%]	FLOPs [10 ⁹]	Parameters [10 ⁶]
1	76	86.28	0.5	0.5
2	40	87.72	0.5	0.5
3	28	86.06	0.5	0.5
4	22	87.60	0.5	0.5

Table 4.6.: Varying deepening factors for Eq-WRN-X-1-C(8-4-1) on CIFAR10 while controlling for parameter count through a change of depth. The deepening factor describes the number of kernels inside each residual block.

the exact experiments performed in [ZK17].

Analysis of both kernel layout and deepening factor, as detailed in Tables 4.5 and 4.6, shows that a 3×3 kernel layout is most effective, not only for the original CNN version but also for our equivariant implementation. This is unsurprising since the architecture is very similar, and the skip connections probably enforce this structure. In section 4.2, our search for the optimal block content is expanded to include various equivariant convolutions and skip connections.

4.1.3. Scaling with Eq-WRN

We have successfully established a robust symmetry group structure and an effective kernel layout for Eq-WRN. In this section, we do a systematic analysis of Eq-WRN

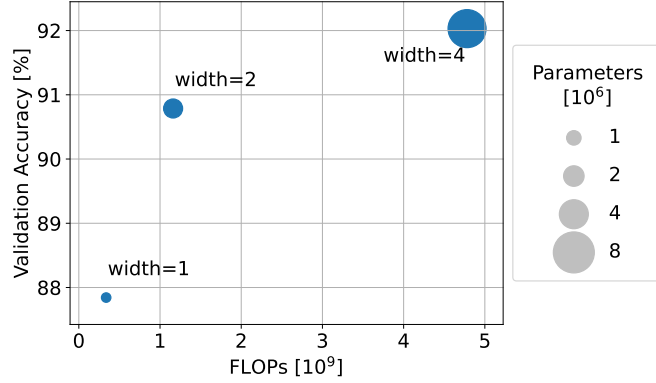


Figure 4.2.: Varying number of initial effective channel sizes for Eq-WRN-34-X-C(8-4-1) on CIFAR10. Increasing the effective channel size leads to higher performance but also increases the computational costs.

scaling behavior. In the experiment on the dihedral group, we have already seen that Eq-WRN-22-6 outperforms Eq-WRN-16-4. However, a comprehensive exploration is required to quantitatively assess how individual modifications in the network’s depth and width affect Eq-WRN’s performance and computational costs.

To answer this question, we initially stepwise increase the width of Eq-WRN-34-1. Our experiment reveals that adding width leads to a performance increase but with a diminishing return on performance for the FLOPs per image. There is a clear trade-off between computational costs and performance visible in Figure 4.2. Given that widen factor = 2 seems like a good compromise between performance and computational costs, we use that for our depth scaling experiments. In this experiment, we stepwise increased the depth of Eq-WRN-16-2. Notice the minimum increase of depth is 6, given the design of the network. We found that the performance increases in a deeper network, but there seems to be a saturation point at depth = 28, as can be seen in Figure 4.3. After that, the performance stagnates or even deteriorates. In comparison to width scaling, depth scaling is less computationally expensive. These results indicate the existence of scaling laws for ECNNs, similar to those reported for CNNs in [TL20]. We did not study the resolution scaling in this section, as increasing the image size for Eq-WRN quickly becomes prohibitively expensive. In the next section, we develop a new baseline architecture for ECNNs, which could be much easier to scale.

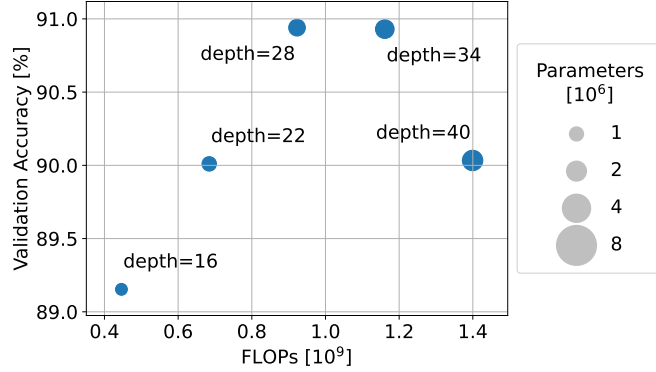


Figure 4.3.: Varying depth levels for Eq-WRN-X-2-C(8-4-1) on CIFAR10. Increasing the depth leads to higher performance up to a point but also increases the computational costs. Increasing the depth further than $n = 28$ leads to stagnation or even performance loss.

4.2. ECNN Baseline Architecture

This section details the development of our optimal ECNN architecture, focusing on balancing performance and computational efficiency. We present a sequential approach: initially demonstrating the progressive performance improvement through the Neural Architecture Search, then establishing a robust baseline architecture, and finally conducting an ablation study to discern critical architectural features.

4.2.1. Neural Architecture Search

To illustrate the effectiveness of our approach, we run a NAS for multiple datasets (ISIC2019, CIFAR10, Galaxy10 DECals, MNIST rot). The ISIC2019 [SH21] dataset presents a challenging task of classifying eight different types of skin lesions. For each dataset, we generate 100 architectures so-called trials, with 25 initial trials using the quasi-random Sobol generator to ensure the broad exploration of the search space, followed by 75 trials using multiobjective full Bayesian optimization. For the Bayesian optimization, we use the standard hyperparameter from Ax, except that we set the number of samples to 256 and a warm-up phase to 512 steps to speed up the generation.

As illustrated in Figure 4.4, the optimization model significantly refines the architecture over time after the initial 25 quasi-random trials. The best architectures exhibit not only comparable or superior performance to the Eq-WRN-16-4 but also improved computational efficiency. Note Eq-WRN-16-4 always uses the optimal group layout

for the given dataset. High computational efficiency is especially relevant for handling large image sizes or use scaling to improve performance even further when a higher computing budget is available.

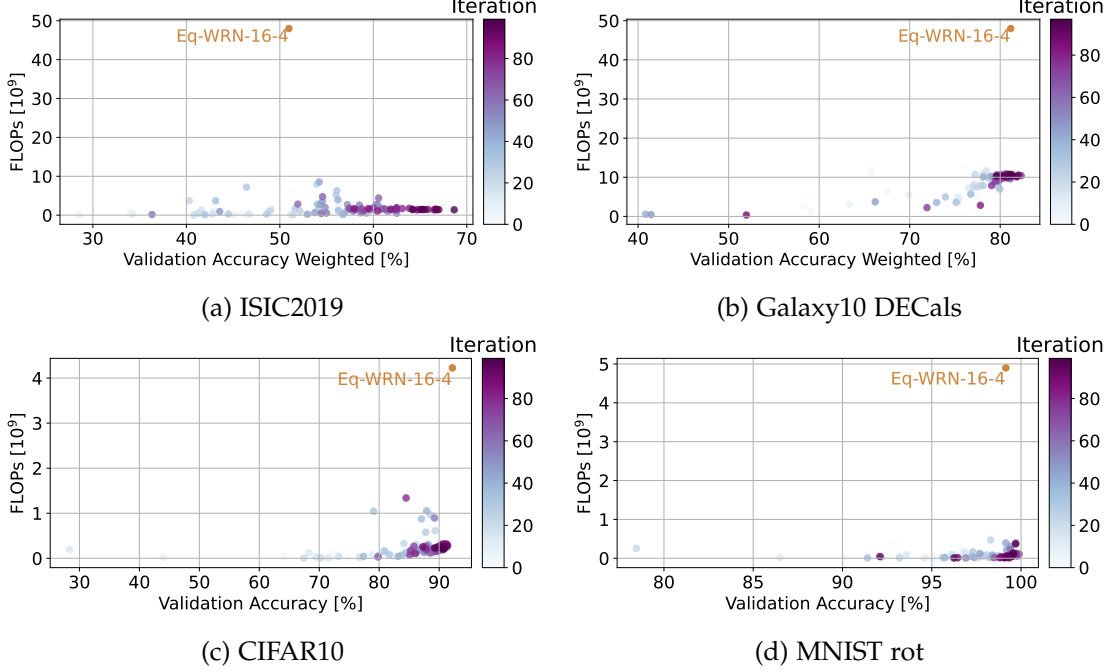


Figure 4.4.: Results of Neural Architecture Search on different datasets. The first 25 trials are quasi-random to explore the architectural design space. Followed by 75 multiobjective full Bayesian optimization trials.

4.2.2. Select and Analyze Eq-NASNet

The task of selecting an optimal baseline architecture from 400 generated candidates required a detailed selection and evaluation process.

Select and Compare Initially, we focus on pinpointing the common features shared by the most efficient and performant architectures. We find several commonalities across the Pareto optimal architectures across the four different datasets. For instance, the lifting convolution consistently used a kernel size 5 and a dihedral group with at least 8 rotations. However, preferences for convolutional operations and internal group configurations vary, reflecting the diverse symmetry levels in the datasets. Since the optimal baseline architecture has to be compatible with processing large images,

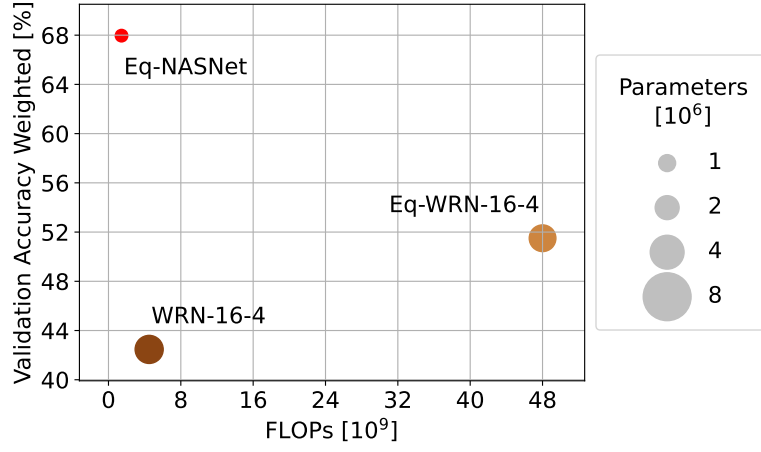
Stage i	Operation F_i, K_i	Stride S_i	Group G_i	Representations R_i	SkipOp SO_i	Layers L_i
0	Conv, 5x5	1	D_4	2	-	1
1	MBConv6, 5x5	2	D_4	8	identity	1
2	MBConv6, 5x5	2	D_4	22	identity	2
3	MBConv6, 5x5	1	C_2	88	identity	2

Table 4.7.: Our baseline architecture **Eq-NASNet** found with NAS on ISIC2019. We change the Squeeze and Excitation ratio of the last block to 0.0 to reduce performance variance and improve the performance on other datasets.

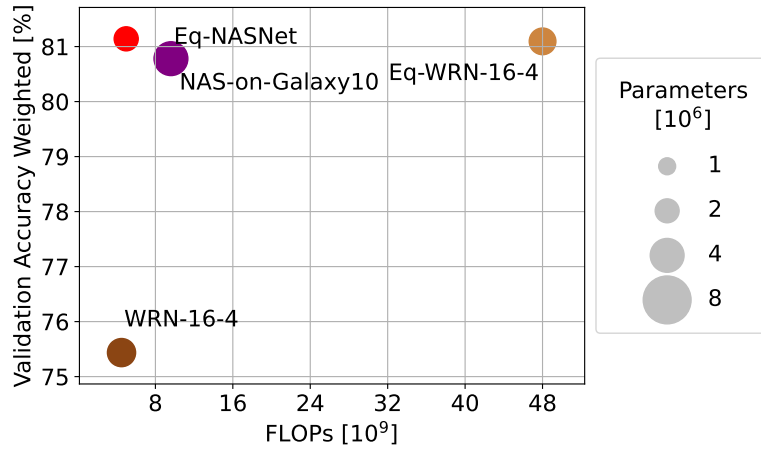
we discard the architectures from CIFAR10 and MNIST rot. Further, the optimization engine found more efficient architectures for ISIC2019 than for Galaxy10 DECals. Therefore, to arrive at the ECNN baseline architecture, we combine the characteristics of five Pareto optimal ISIC2019 NAS architectures. This architecture is subsequently called Eq-NASNet. See Table 4.7 for details about the architecture. Eq-NASNet has a high dropout rate of 0.5, likely due to the need for more regularization in equivariant architectures or the complexity of the ISIC2019 dataset.

Cross-dataset Evaluation To see if Eq-NASNet performs well across a wide variety of datasets (ISIC2019, Galaxy10 DECals, MNIST rot, CIFAR10, MNIST), we compare it against Eq-WRN-16-4, WRN-16-4 and, if available the most performant architecture found by the NAS on that dataset. We reduce the dropout rate for the other datasets for faster training and slightly improved performance. Figure 4.5 presents the performance of Eq-NASNet, surpassing Eq-WRN-16-4 and WRN-16-4 in all datasets except CIFAR10, where it slightly lags behind the larger Eq-WRN-16-4. Eq-NASNet stands out for its parameter efficiency, translating to lower storage costs and comparable FLOPs per training image to WRN-16-4.

4. Results

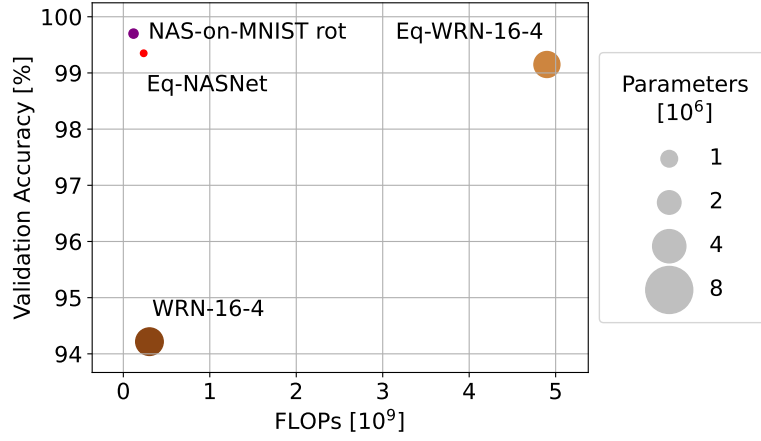


(a) ISIC2019. Eq-NASNet outperforms WRN-16-4 and Eq-WRN-16-4 in performance, parameters, and FLOPs.

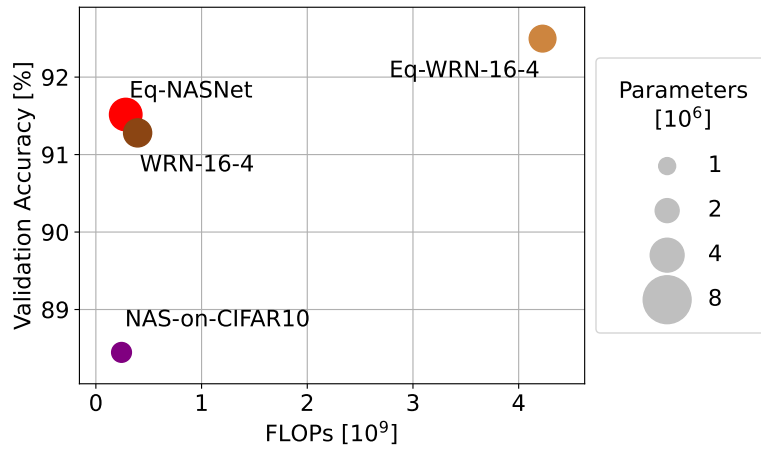


(b) Galaxy10 DECals. Details: width coefficient = 2, dropout rate = 0.2. Eq-NASNet outperforms WRN-16-4 and Eq-WRN-16-4 in performance, parameters, and FLOPs.

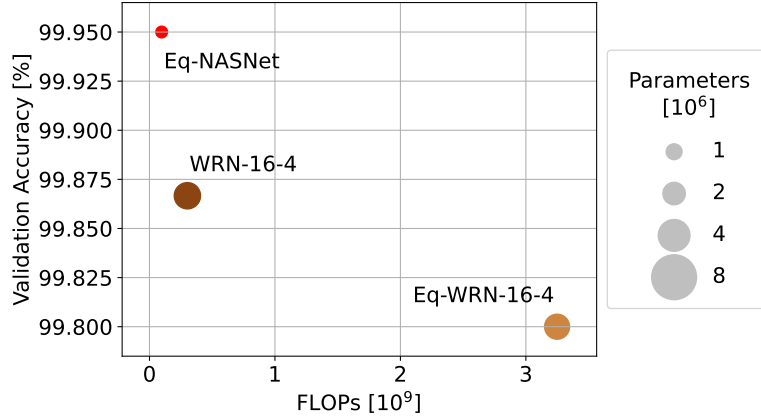
4. Results



(c) MNIST rot. Details: dropout rate = 0.2, increase group D16, no restriction. Eq-NASNet is roughly on par with the specialized NAS-on-MNIST rot and outperforms WRN-16-4.



(d) CIFAR10. Details: dropout rate = 0.2, width coefficient = 1.5. Eq-NASNet outperforms WRN-16-4 and Eq-WRN-16-4 in FLOPs but Eq-WRN-16-4 achieves a better performance with higher computational costs.



(e) MNIST. Details: dropout rate = 0.0. Eq-NASNet outperforms WRN-16-4 and Eq-WRN-16-4 in performance, parameters, and FLOPs.

Figure 4.5.: Comparison of Eq-NASNet, Eq-WRN-16-4, and WRN-16-4 on different datasets. Eq-NASNET is baseline architecture from Table 4.7. If we applied changes to the baseline architecture, we noted them in the subcaptions.

Ablation Study To understand which architectural details are important for optimal performance, an ablation study is conducted on convolutional operation types, kernel sizes, and the Squeeze and Excitation ratio (SE ratio) in the last block using the ISIC2019 dataset. To guarantee a fair comparison, we keep the computation costs roughly constant by adjusting the width of the network. The results are summarized in Table 4.8. This study underscores the superiority of MBconv compared to the other convolution types. MBconv can provide high performance at low computational costs by only performing the filtering operation in a high-dimensional space. A higher SE ratio does not substantially enhance performance and introduces greater variance. Striving for a baseline architecture that performs consistently well across multiple datasets with minimal performance variance, we set the SE ratio of the last block to 0. A kernel size of 3 degrades the performance moderately, which suggests the existence of interpolation artifacts.

Architecture	Change [%]	Validation Accuracy Weighted [%]
Baseline ($K_i = 5$, $F_i = \text{MBconv}$, $\text{SE}_3=0.75$)	-	66.67
$K_i = 3$	-2.7	63.97
$F_i = \begin{cases} \text{Conv} \\ \text{Dconv} \end{cases}$	-11.88 -12.01	54.79 54.66
$\text{SE}_3 = 0.0$	-0.02	66.65

Table 4.8.: Ablation study on ISIC2019 for the type of convolution operation and kernel size in every block and the Squeeze and Excitation ratio in the last block. FLOPs are kept approximately constant by adjusting the initial effective channel size.

4.3. Scaling Laws for ECNNs

We have already looked at the individual scaling behavior in width and depth of Eq-WRN in this chapter. However, the high computational demand of Eq-WRN makes scaling resolution prohibitively expensive, especially if it is combined with width scaling. With the small and efficient baseline ECNN architecture Eq-NASNet, we can extend our analysis to resolution and compound scaling.

Initially, we stepwise increase the width, depth, and resolution separately. For depth scaling, we experiment with increasing the number of blocks and layers in each block. Concretely, we clone block 2, which turns out to be essential for reducing the feature map size when we deal with larger images. The layer setup is indicated by d . So $d=1-2-2-2$ would be Eq-NASNet with 4 blocks and no change to the number of layers in each block. The individual scaling experiments are followed by a combined scaling of all three dimensions. For combined scaling, we fix three different starting configurations and scale them up in width. All experiments are conducted on the ISIC2019 dataset. To reduce the variance in the final result, we change to a fixed learning rate scheduler. Additionally, we increase the number of epochs to 150 to also allow the larger networks to converge. Unless otherwise specified, the base resolution rate is 96×96 .

In Figure 4.6, we can observe the scaling behavior for all three individual scaling dimensions. Width scaling shows increased performance but at a diminishing return

4. Results

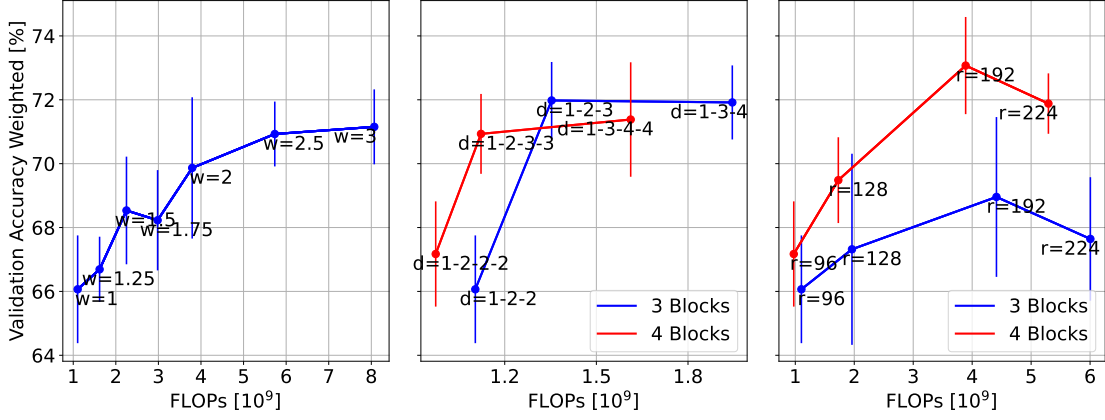


Figure 4.6.: Scaling of Eq-NASNet on ISIC2019 in terms of width (left), depth (middle), resolution (right). The error bars indicate the standard deviation across 5 independent runs.

on computational investment. Similar to our observations in Eq-WRN and traditional CNNs. Depth scaling exhibits a comparable overall trend. However, it achieves enhancements similar to width scaling but at a notably lower computational expense. As for Eq-WRN, there seems to be no additional improvement possible after reaching a saturation point. A possible reason for this is the problem with the skip connections. In CNNs, skip connections provide a direct gradient path. However, for ECNNs, if we reduce the symmetry group, the gradient path is partially obstructed by a 1×1 convolution at that level. The convolution ensures that the representations are compatible. The effectiveness of resolution scaling in Eq-NASNet is significantly influenced by the number of blocks in the network. When the network comprises only 3 blocks, the benefits of resolution scaling are relatively modest. However, as the number of blocks increases, resolution scaling becomes more effective, likely due to the expanded receptive field in a deeper network. A noteworthy observation is the performance drop at a resolution of 224×224 , leading to two potential hypotheses: firstly, a deeper network with additional blocks might be better suited for even larger image sizes, and secondly, enhanced regularization strategies, such as increased dropout rates, could facilitate the learning process with larger images. While we did not test these hypotheses, they present interesting avenues for future research.

The integration of various scaling methods has been shown to yield optimal results in CNNs. Our study extends this finding to ECNNs, demonstrating that a combination of width, depth, and resolution scaling produces the most effective performance en-

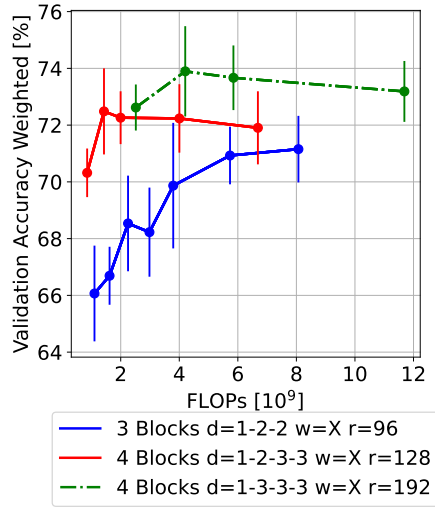


Figure 4.7.: Compound scaling of Eq-NASNet on ISIC2019. The points in the same color are the initial starting configuration width-scaled. The error bars indicate the standard deviation across 5 independent runs.

hancements. Figure 4.7 illustrates this effect through the width scaling of three distinct starting configurations. This comprehensive approach underscores the versatility and efficacy of combined scaling strategies in optimizing ECNN performance.

5. Comparison to Leading Image Classification Architectures

This chapter evaluates the performance of our proposed architecture, Eq-NASNet with 4 Blocks, against leading models in image classification: EfficientNet [TL20] and Vision Transformer (ViT) [Dos+21]. The comparison involves models with both randomly initialized and ImageNet pre-trained weights^{1,2}. EfficientNet serves as a good benchmark due to its optimization for both performance and compute with a search space similar to Eq-NASNet. Additionally, we add ViT to our comparison, given its recent successes in the field. We include pre-trained networks in our analysis because, particularly in medical image analysis and scenarios with limited data, starting with pre-trained weights is a common practice. We compare these five models (Eq-NASNet, EfficientNet, EfficientNet pre-trained, ViT, ViT pre-trained) on two challenging medical datasets, for efficiency, under low-data regimes, and against adversarial attacks. Hyperparameter optimization (HPO) for learning rate, weight decay, and learning rate scheduler is conducted for each model variant using Ray 2.8.1 [Lia+18] and the Ax optimization engine with 25 trials per dataset. Details on the HPO are provided in the Abstract A.2. Unless otherwise noted, all models are trained with 128×128 resolution.

¹https://pytorch.org/hub/nvidia_deeplearningexamples_efficientnet

²https://pytorch.org/vision/main/models/generated/torchvision.models.vit_b_16.html#torchvision.models.vit_b_16

5.1. Medical Data

Considering the high stakes involved in medical image analysis, it is important to leverage the increased inherent symmetries in medical data. Medical datasets often present unique challenges, such as high variability and significant class imbalances, where improved generalization of ECNNs can be helpful. In this section, we assess the performance of Eq-NASNet on two challenging medical datasets: ISIC2019 and the second DeepDRiD challenge [Liu+22]. These datasets focus on dermatoscopy and diabetic retinopathy image quality assessment, respectively. We compare Eq-NASNet with the four model variants, evaluating their effectiveness in these specialized domains.

Eq-NASNet outperforms all randomly initialized models and competes closely with or surpasses pre-trained models (see Tables 5.1 and 5.2). Specifically, on ISIC2019, Eq-NASNet achieves more than a 3% improvement in weighted accuracy over all other models and even surpasses the highly augmented best submitted single model by Gessert et al. [Ges+19]. This not only highlights Eq-NASNets strengths but also underscores the potential of ECNNs in advancing medical image analysis.

Model	Test Accuracy Weighted [%]	Parameters [10^6]	FLOPs [10^9]
Gessert et al. [Ges+19]	68.8 (§0.7)	40.76	19.97
EfficientNet	39.76 (§1.52)	4.01	0.14
$\hookrightarrow$ pre-trained	62.97 (§3.33)	4.01	0.14
ViT	43.51 (§1.89)	85.69	5.5
$\hookrightarrow$ pre-trained	66.34 (§1.64)	85.69	5.5
Eq-NASNet	69.69 (§0.07)	0.66	1.7

Table 5.1.: Performance comparison of our model on the ISIC2019 dataset across 5 independent runs. Our models are trained without any augmentation. The pre-trained networks were pre-trained on ImageNet-1k and subsequently fine-tuned on ISIC2019. For comparison, the best single model submitted by Gessert et al., an EfficientNet B6 with a resolution of 528×528 , was trained using extensive augmentation.

Model	Test Accuracy [%]	Parameters [10^6]	FLOPs [10^9]
Tummala et al. [Tum+23]	73.5	120	5.3
EfficientNet	69.25 (§3.6)	4.01	0.14
$\hookrightarrow$ pre-trained	73.65 (§1.07)	4.01	0.14
ViT	68.92 (§1.36)	85.69	5.5
$\hookrightarrow$ pre-trained	74.19 (§0.93)	85.69	5.5
Eq-NASNet	73.11 (§1.28)	0.66	1.7

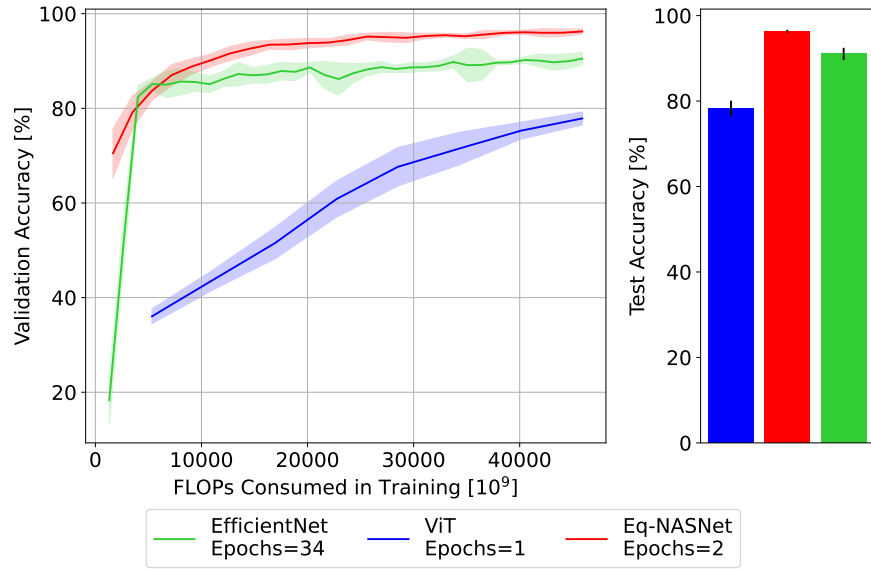
Table 5.2.: Performance comparison of our model on the second challenge DeepDRiD dataset across 5 independent runs. Our models are trained with mild color augmentation. The pre-trained networks were pre-trained on ImageNet-1k and subsequently fine-tuned on DeepDRiD. For comparison, the best published single model by Tummala et al., an EfficientNetV2-L with resolution 224×224 , was trained using extensive augmentation.

5.2. Efficiency

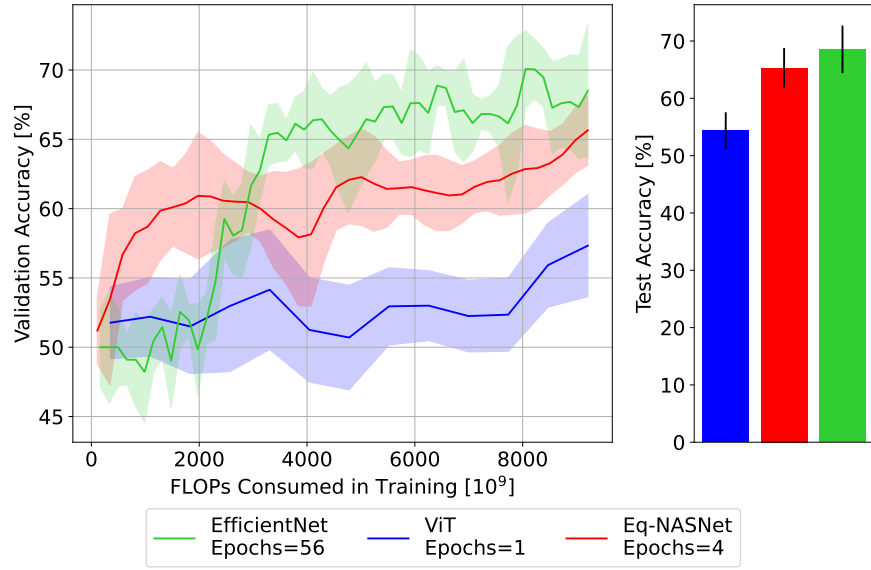
Efficiency in neural networks is critical, especially when considering deployment in real-world applications where resources are often limited. For example, in the case of medical data, it is often not possible to send the data to a compute cluster with specialized hardware due to privacy issues. In this section, we compare Eq-NASNet, EfficientNet, and ViT in terms of computational requirements. From the previous section, we know that Eq-NASNet is more parameter efficient than EfficientNet ($\approx 6\times$) and ViT ($\approx 128\times$), as shown in Tables 5.1 and 5.2. While it uses fewer FLOPs than ViT ($\approx 3\times$), it requires more than EfficientNet ($\approx 12\times$) for a single training image. However, Eq-NASNet reaches convergence in fewer epochs than EfficientNet on both datasets.

For a fair efficiency comparison, we conduct an experiment with an equal computation training budget (measured in FLOPs). The training budget is set based on the average FLOPs consumed in training EfficientNet to convergence, which we calculate over three independent runs. Subsequently, both models are trained within this predetermined FLOP budget. We increase the learning rate and decrease the dropout rate of Eq-NASNet to match the EfficientNet optimal hyperparameters. This gives a slight edge to EfficientNet in the comparison, but apart from running another HPO, there is no systematic way to determine those parameters for this new training setting. For good measure, we add ViT but use originally found hyperparameters for ViT since these result in slightly better performance.

As illustrated in Figure 5.1, Eq-NASNet outperforms both EfficientNet and ViT on the Blood dataset. However, it is surpassed by EfficientNet on the DeepDRiD dataset. This outcome aligns with expectations, considering the relatively small size of the DeepDRiD dataset, comprising only 1200 training images. Consequently, during its training, Eq-NASNet is exposed to approximately 5000 images, which may not be adequate for fully optimizing all the randomly initialized weights.



(a) Blood



(b) DeepDRiD

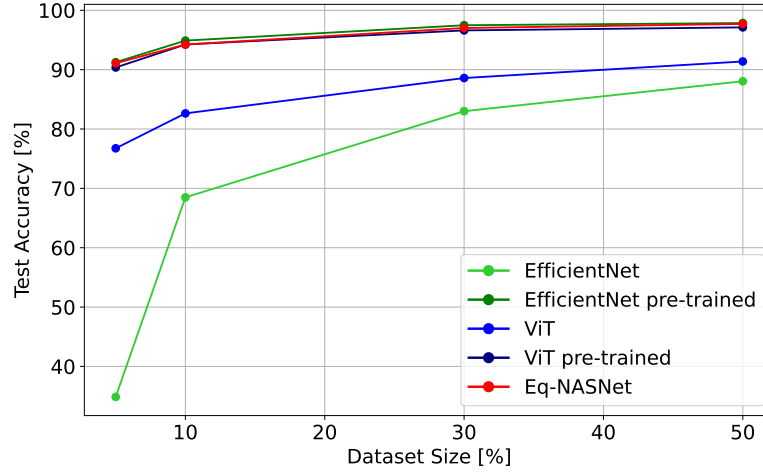
Figure 5.1.: Performance comparison of the randomly initialized models under a given computational budget for the Blood and DeepDRiD datasets. The training budget is determined on the average FLOPs consumed in training EfficientNet to convergence. Standard deviations are shown over 5 independent runs.

5.3. Low Data Regime

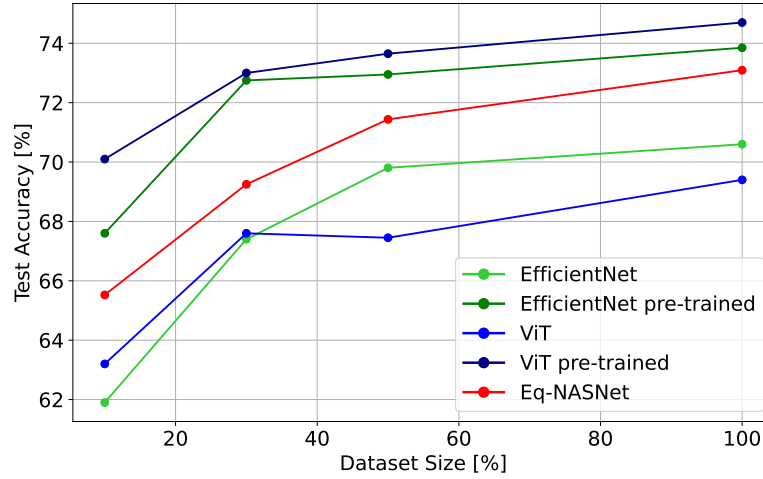
In recent years, the trend in machine learning has gravitated towards increasingly larger models, necessitating correspondingly extensive training datasets. However, in many domains, particularly in medical imaging, acquiring large datasets can be prohibitively expensive or practically unfeasible. Medical datasets often have fewer than 100 images per class and are typically characterized by significant class imbalances. While healthy subject images are more readily available, acquiring sufficient images of diseased classes poses a substantial challenge.

A common strategy to circumvent the scarcity of large datasets is the utilization of pre-trained models, which are then fine-tuned on the smaller, specific dataset. This approach's effectiveness largely depends on the degree of domain shift. Notably, even with significant shifts, such as from natural image domains to medical imaging, pre-training can be beneficial due to the similarity in low-level feature extraction processes. Nonetheless, this method has limitations, especially when the available data per class is extremely limited. The built-in rotational symmetric bias of ECNNs makes them potentially a more efficient pattern extractor since fewer parameters need to be adjusted.

In Figure 5.2, it can be seen that our Eq-NASNet outperforms all non-pre-trained networks when we gradually reduce the size of the training dataset. It has comparable performance to the pre-trained networks on the Blood dataset but is slightly outperformed on the difficult DeepDRiD challenge. As already mentioned, the DeepDRiD training dataset only consists of 1200 images, which is further reduced in this task. Learning all the pattern extraction steps plus good generalization is extremely difficult from very limited data. We hypothesize that a pre-trained Eq-NASNet would perform even better than other pre-trained networks. However, we did not investigate this further due to limited computational resources. This could be a very interesting direction for future research.



(a) Blood



(b) DeepDRiD

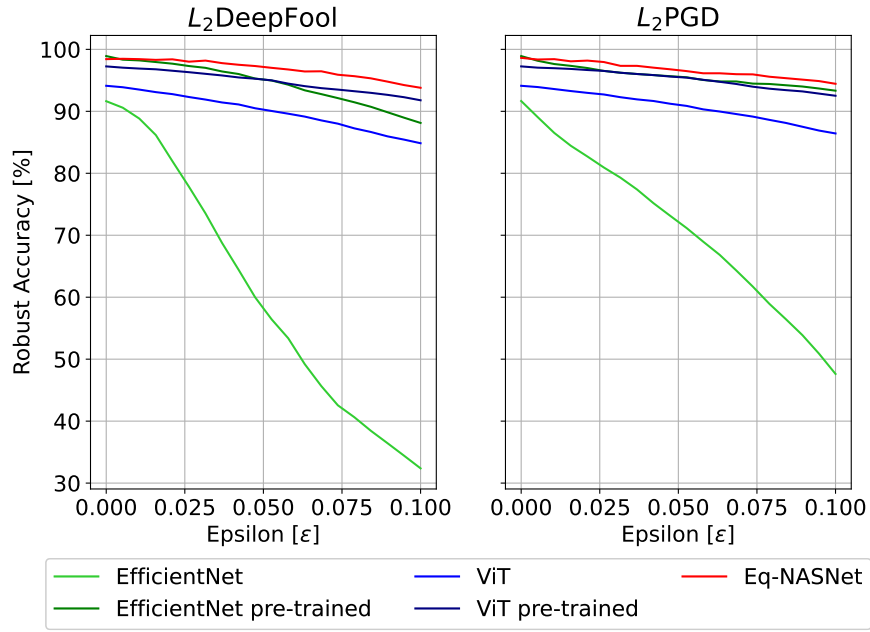
Figure 5.2.: Performance comparison under reduced training dataset size for the Blood and DeepDRiD datasets.

5.4. Adversarial Attacks

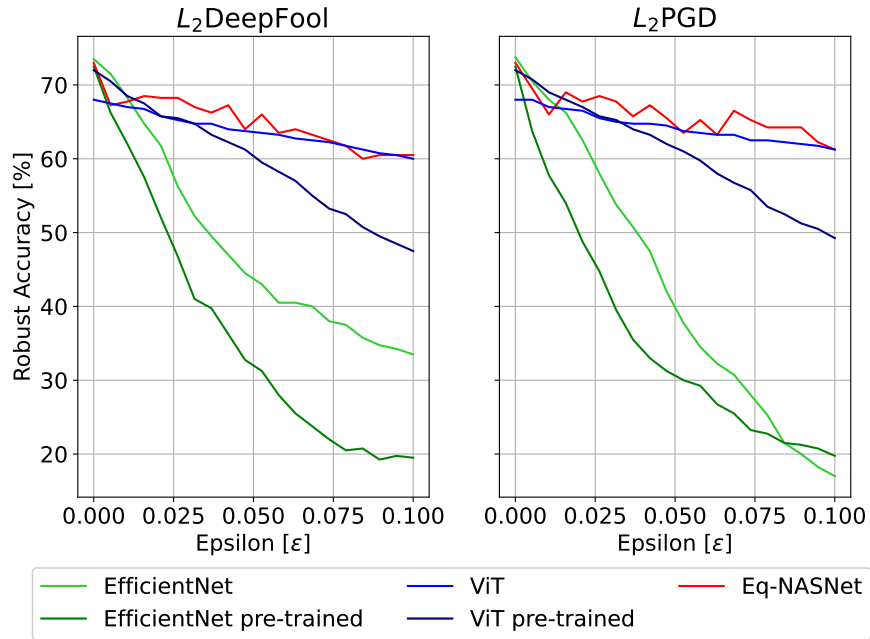
It is well known that modern neural networks are vulnerable to human-imperceptible adversarial perturbations [ZH18]. In the context of image classification, that translates to perturbing some or all of the pixels in the input image in a predefined small range (ϵ) such that it potentially causes a misclassification. The success of such an adversarial attack is determined by whether the network erroneously classifies the manipulated image while the perturbations stay within the predetermined ϵ boundary. Naturally, the larger the ϵ range is, the easier it is to find an adversarial image. Adversarial attacks are generally divided into two categories: black-box and white-box attacks. White-box attacks exploit the network’s gradient information, while black-box attacks often involve creating a surrogate model to perform the attack. Black-box attacks tend to be more computationally demanding and less effective compared to their white-box counterparts.

Our study focuses on two prominent white-box, gradient-based attacks, utilizing Foolbox 3.3.3 for implementation [RBB17; Rau+20]. We choose the DeepFool attack for its efficacy in low ϵ ranges, slightly outperforming the well-known Fast Gradient Sign Method (FGSM) [Zha+23]. Additionally, we employ the Projected Gradient Descent (PGD) attack, known as a robust benchmarking tool and one of the strongest first-order attacks [Zha+23]. In our evaluation, we use the L_2 metric to measure the ϵ distance and assess each model’s performance at 20 linearly spaced ϵ values ranging from 0.0 to 0.1. We adopt the literature’s typical evaluation metric of robust accuracy. Robust accuracy is the accuracy after the perturbation. In other words, how many images are classified correctly after the attack.

The results, depicted in Figure 5.3, reveal that our Eq-NASNet either matches or exceeds the performance of the benchmark models under these attacks. Notably, Eq-NASNet shows impressive resilience against attacks even at higher ϵ values, despite not being pre-trained. This is in contrast to the non-pre-trained CNN model, EfficientNet, which demonstrates significantly weaker performance under the same conditions. This suggests that Eq-NASNet may be learning more robust pattern representations than its CNN counterparts. Robustness against adversarial attacks increases trust in machine learning models, which is especially important for high-stakes environments like medical data interpretation and diagnosis.



(a) Blood



(b) DeepDRiD

Figure 5.3.: Performance comparison under different adversarial attacks for increasing ϵ epsilon distance for the Blood and DeepDRiD datasets.

6. Conclusion

In this work, we address the promising yet underexplored domain of Equivariant Convolutional Neural Networks, focusing on their design space and practical application potential. ECNNs, extend the translational symmetries of traditional CNNs by incorporating rotations and reflections. These networks have shown potential in improving sample efficiency and generalization. However, their widespread implementation in real-world applications has been limited due to a lack of comprehensive understanding of the design space and the absence of established baseline architectures.

Our main contributions through this work include the development and validation of Eq-NASNet, a novel ECNN architecture optimized for both computational efficiency and performance. Eq-NASNet notably outperformed established models like EfficientNet and Vision Transformer in challenging medical datasets. Additionally, Eq-NASNet excelled in low-data scenarios and exhibited robustness against adversarial attacks while maintaining high training efficiency. Further, we successfully demonstrated the possibility of increasing performance further by scaling up width, depth, and resolution, filling a significant gap in the current understanding of ECNNs.

Looking towards the future, several research directions appear promising. Domain shift remains a significant challenge for current machine learning systems. The greater generalization of ECNNs in general and Eq-NASNet, in particular, might mitigate this problem and facilitate a more widespread use of artificial intelligence. Another interesting avenue is the exploration of pre-training strategies for Eq-NASNet. Given the difficulty and costs of obtaining large, annotated datasets, the ability of Eq-NASNet to perform effectively in low-data environments is particularly valuable. Pre-training could further enhance this capability, enabling Eq-NASNet to deliver high performance even with very limited data.

In conclusion, this work not only deepens the understanding of ECNNs but also establishes Eq-NASNet as a robust baseline architecture for future research in this promising field. The implications of our findings are far-reaching, particularly in fields like healthcare and satellite imagery, where leveraging increased symmetries is vital. Our findings could set new benchmarks in the development and application of these spe-

6. Conclusion

cialized neural networks that are both more efficient and effective.

A. Appendix

A.1. Trainings Setup for Different Datasets

We use the cross entropy loss for training all datasets and adopted the following training setups:

CIFAR10 For CIFAR10 [Kri09], we use a similar training setup as [ZK17]. The optimizer is SGD with learning rate 0.1, momentum 0.9, Nesterov momentum, and weight decay $5 \cdot 10^{-4}$. We multiply the learning rate with 0.2 every 60 epochs. In contrast to [ZK17], we report our results unless otherwise stated for 100 epochs to reduce the computational costs.

MNIST rot For MNIST rot [Lar+07], we use a similar training setup as [WC19]. Split the data into 10000 + 2000 training and validation samples and train for 40 epochs with batch size 64 and initial learning rate of 0.015 with an exponential decay of 0.7 and weight decay of $5 \cdot 10^{-4}$.

Galaxy10 DECals For the Galaxy10 DECals dataset [LB18], we use the Adam optimizer, with an initial learning rate of 0.01 and a weight decay of $1 \cdot 10^{-6}$. We train for 50 epochs with a batch size of 256. We employ a multi-step learning rate scheduler, reducing the learning rate by 0.02 at the milestone of 25 epochs. To address the class imbalance, we weighted the loss function with the inverse class distributions. The images are processed at a resolution of 108 pixels. The dataset split follows a 9:1:1 ratio for training, validation, and test sets. No data augmentation is applied apart from resizing.

ISIC2019 For the ISIC2019 dataset [SH21], we use the Adam optimizer with a learning rate of 0.005 and a weight decay of $1 \cdot 10^{-5}$. We train for 120 epochs with a batch size of 64. We employ a multi-step learning rate scheduler, reducing the learning rate by 0.5 at milestones of 35, 75, 90, and 105 epochs. To address the class imbalance, we weighted the loss function with the inverse class distributions. The images are processed at a resolution of 128 pixels. An early stopping mechanism is used, focusing

on maximizing the weighted validation accuracy, with a patience of 20 epochs. The dataset is split in a 9:1:1 ratio for training, validation, and test sets. No data augmentation is applied apart from resizing. The challenge dataset is a combination of the following databases: BCN 20000[Com+19], HAM10000 [TRK18], MSK [Cod+18].

DeepDRiD For the DeepDRiD dataset [Liu+22], we utilize the Adam optimizer with a learning rate of 0.001 and a weight decay of $1 \cdot 10^{-5}$. The training is conducted over 100 epochs with a batch size of 64. We do not use a learning rate scheduler for this dataset. The images are processed at a resolution of 128 pixels. We incorporate an early stopping mechanism, which is designed to minimize the validation loss, with a patience of 20 epochs. The dataset split adheres to the original challenge configuration, ensuring consistency with its evaluation standards. In terms of data augmentation, we apply short side-center cropping, color jitter (with adjustments in brightness, contrast, saturation, and hue), and random affine transformations including rotations and translations, along with a random grayscale conversion.

Blood For the Blood dataset [Ace+19], we utilize the Adam optimizer with a learning rate of 0.005 and a weight decay of $1 \cdot 10^{-5}$. The training is conducted over 100 epochs with a batch size of 64. To address the issue of class imbalance in this dataset, we apply weights to the loss function derived from the inverse class distributions. The images are processed at a resolution of 128 pixels. We incorporate an early stopping mechanism, which is designed to minimize the validation loss, with a patience of 20 epochs. The dataset is divided in a 7:1:2 ratio for the training, validation, and test sets, following the split strategy as outlined in [Yan+23].

A.2. Hyperparameter Optimization

To guarantee a fair comparison, we must find good hyperparameters for every model. For our study, we focus on optimizing three key hyperparameters: learning rate, weight decay, and the configuration/existence of the learning rate scheduler. This optimization process is aimed at maximizing the (weighted) validation accuracy of our models.

The parameters we chose to optimize include:

- **Learning Rate:** We vary the learning rate within a range of $5 \cdot 10^{-5}$ to $1 \cdot 10^{-2}$, leveraging the *loguniform* distribution to explore this scale effectively.
- **Weight Decay:** The weight decay parameter is explored within the bounds of $1 \cdot 10^{-7}$ to $1 \cdot 10^{-3}$, also utilizing the *loguniform* distribution.

- **Learning Rate Scheduler Parameters:** The patience parameter of the scheduler, which dictates the reduction of learning rate on plateaus, is set to vary between $0.05 * \text{epochs}$ and the total number of epochs. This lets the optimization model effectively turn off the learning rate scheduler. The factor by which the learning rate is reduced was explored within a range of 0.01 to 0.5, following a *loguniform* distribution.

For the optimization process, we utilize the Ray framework with the Async Successive Halving algorithm [Li+20] in conjunction with the Ax optimization engine. This combination allows us to explore the hyperparameter space efficiently. We schedule 25 trials per dataset, which is sufficient for convergence for all datasets after manual inspection of the optimization process. Notably, we set the grace period for Ray early stopping at 50% of the number of epochs trained, providing a balanced approach between exploration and exploitation in our optimization strategy.

Abbreviations

CNN Convolutional Neural Network

ECNN Equivariant Convolutional Neural Network

Eq-WRN Equivariant Wide Residual Network

FGSM Fast Gradient Sign Method

FLOPs Floating Point Operations

HPO Hyperparameter optimization

irreps irreducible representation

NAS Neural Architecture Search

PGD Projected Gradient Decent

SE ratio Squeeze and Excitation ratio

ViT Vision Transformer

WRN Wide Residual Network

List of Figures

2.1. Illustration of rotation equivariance processing with an ECNN from a theoretical perspective inspired by [WHS18] using the human eye [Com23a] as an example. $f_{h_j}^{(i)}, k_{h_j}^{(i)}$ represent the feature map $\mathbf{f}$ and kernel $\mathbf{k}$ in layer i endowed with the group action h_j . Filter kernels in the same color share weights since these are the same kernels but rotated by the group action h . The orientation of the symbols illustrates the rotation of the kernel.	9
3.1. Images can exhibit symmetries at different scales. Natural images like the sunflower field show rotational symmetries primarily at the local level, like in the sunflower heads. The rotational symmetry is lost at the larger scale due to a clear up-and-down orientation. For many medical images, like the histopathology image, the orientation should not impact the classification.	13
3.2. Illustration of our factorized search space inspired by [Tan+19]. The network layers are organized into distinct blocks, each with specific input resolutions, groups, and effective channel sizes. A block consists of a possible group restriction (Res) and a variable number of layers. All layers of a block are the same except the first. The first convolution in the layer implements the stride and increase (C_i) of the effective channel size for the block. The figure illustrates the adaptability of our architecture skeleton, emphasizing the search for operations and connections within a single layer and the repetition across the L layers.	20
3.3. Illustration of Pareto frontier for multiobjective optimization in performance and computational costs. The goal is to minimize FLOPs and maximize accuracy.	24
4.1. Varying initial rotation levels for Eq-WRN-16-4-C(X-X/2-1) on CIFAR10.	26
4.2. Varying number of initial effective channel sizes for Eq-WRN-34-X-C(8-4-1) on CIFAR10. Increasing the effective channel size leads to higher performance but also increases the computational costs.	30

4.3. Varying depth levels for Eq-WRN-X-2-C(8-4-1) on CIFAR10. Increasing the depth leads to higher performance up to a point but also increases the computational costs. Increasing the depth further than $n = 28$ leads to stagnation or even performance loss.	31
4.4. Results of Neural Architecture Search on different datasets. The first 25 trials are quasi-random to explore the architectural design space. Followed by 75 multiobjective full Bayesian optimization trials.	32
4.5. Comparison of Eq-NASNet, Eq-WRN-16-4, and WRN-16-4 on different datasets. Eq-NASNET is baseline architecture from Table 4.7. If we applied changes to the baseline architecture, we noted them in the sub-captions.	36
4.6. Scaling of Eq-NASNet on ISIC2019 in terms of width (left), depth (middle), resolution (right). The error bars indicate the standard deviation across 5 independent runs.	38
4.7. Compound scaling of Eq-NASNet on ISIC2019. The points in the same color are the initial starting configuration width-scaled. The error bars indicate the standard deviation across 5 independent runs.	39
5.1. Performance comparison of the randomly initialized models under a given computational budget for the Blood and DeepDRiD datasets. The training budget is determined on the average FLOPs consumed in training EfficientNet to convergence. Standard deviations are shown over 5 independent runs.	44
5.2. Performance comparison under reduced training dataset size for the Blood and DeepDRiD datasets.	46
5.3. Performance comparison under different adversarial attacks for increasing ϵ epsilon distance for the Blood and DeepDRiD datasets.	48

List of Tables

3.1. Our implementation of an Eq-WRN. The notation for the block $[3 \times 3, 16 \cdot k, g_1]$ in a Layer1 is as follows: 3×3 indicates two equivariant steerable convolutions separated by an activation function and a batch normalization, each with a kernel size of 3. $16 \cdot k$ marks the effective channel size. g_1 represents the group of the regular representations. N is calculated based on the depth.	17
3.2. Details on the options available during the NAS.	21
3.3. Heuristic dataset-dependent upper bounds for stopping trials during the NAS.	23
4.1. Varying restriction levels factors for Eq-WRN-16-4-X on different datasets. FLOPs can not be compared across datasets as they have different image sizes.	27
4.2. Difference between the cyclic and dihedral group for different Eq-WRN sizes. The restriction logic remains the same.	27
4.3. Relationship between kernel size and initial rotation level with different parameter counts for Eq-WRN-16-4-C(X-X/2-1) with on CIFAR10. . . .	28
4.4. Relationship between kernel size and initial rotation level with approximately similar parameter count through width scaling for Eq-WRN-16-4-C(X-X/2-1) with on CIFAR10. When we increased the kernel size from 3×3 to 5×5 , we decreased the width from 4 to 2.5.	28
4.5. Varying kernel layouts for Eq-WRN-16-X-C(8-4-1) on CIFAR10 while approximately controlling for parameter count through increasing the number of initial channels. Kernel layout 3×3 works best for both CNNs and ECNNs.	29
4.6. Varying deepening factors for Eq-WRN-X-1-C(8-4-1) on CIFAR10 while controlling for parameter count through a change of depth. The deepening factor describes the number of kernels inside each residual block.	29
4.7. Our baseline architecture Eq-NASNet found with NAS on ISIC2019. We change the Squeeze and Excitation ratio of the last block to 0.0 to reduce performance variance and improve the performance on other datasets. . .	33

4.8. Ablation study on ISIC2019 for the type of convolution operation and kernel size in every block and the Squeeze and Excitation ratio in the last block. FLOPs are kept approximately constant by adjusting the initial effective channel size.	37
5.1. Performance comparison of our model on the ISIC2019 dataset across 5 independent runs. Our models are trained without any augmentation. The pre-trained networks were pre-trained on ImageNet-1k and subsequently fine-tuned on ISIC2019. For comparison, the best single model submitted by Gessert et al., an EfficientNet B6 with a resolution of 528×528 , was trained using extensive augmentation.	42
5.2. Performance comparison of our model on the second challenge DeepDRiD dataset across 5 independent runs. Our models are trained with mild color augmentation. The pre-trained networks were pre-trained on ImageNet-1k and subsequently fine-tuned on DeepDRiD. For comparison, the best published single model by Tummala et al., an EfficientNetV2-L with resolution 224×224 , was trained using extensive augmentation.	42

Bibliography

- [Ace+19] A. Acevedo, S. Alf  rez, A. Merino, L. Puigv  , and J. Rodellar. “Recognition of peripheral blood cell images using convolutional neural networks.” In: *Computer Methods and Programs in Biomedicine* 180 (2019), p. 105020. issn: 0169-2607. doi: <https://doi.org/10.1016/j.cmpb.2019.105020>.
- [Bal+20] M. Balandat, B. Karrer, D. R. Jiang, S. Daulton, B. Letham, A. G. Wilson, and E. Bakshy. “BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization.” In: *Advances in Neural Information Processing Systems* 33. 2020.
- [CGW18] T. S. Cohen, M. Geiger, and M. Weiler. *Intertwiners between Induced Representations (with Applications to the Theory of Equivariant Neural Networks)*. 2018. arXiv: 1803.10743 [cs.LG].
- [CGW20] T. Cohen, M. Geiger, and M. Weiler. *A General Theory of Equivariant CNNs on Homogeneous Spaces*. 2020. arXiv: 1811.02017 [cs.LG].
- [CLW22] G. Cesa, L. Lang, and M. Weiler. “A Program to Build E(N)-Equivariant Steerable CNNs.” In: *International Conference on Learning Representations*. 2022.
- [Cod+18] N. C. F. Codella, D. Gutman, M. E. Celebi, B. Helba, M. A. Marchetti, S. W. Dusza, A. Kalloo, K. Liopyris, N. Mishra, H. Kittler, and A. Halpern. *Skin Lesion Analysis Toward Melanoma Detection: A Challenge at the 2017 International Symposium on Biomedical Imaging (ISBI), Hosted by the International Skin Imaging Collaboration (ISIC)*. 2018. arXiv: 1710.05006 [cs.CV].
- [Com+19] M. Combalia, N. C. F. Codella, V. Rotemberg, B. Helba, V. Vilaplana, O. Reiter, C. Carrera, A. Barreiro, A. C. Halpern, S. Puig, and J. Malvehy. *BCN20000: Dermoscopic Lesions in the Wild*. 2019. arXiv: 1908.02288 [eess.IV].
- [Com23a] W. Commons. *Fundus photograph of normal right eye*. 2023. URL: https://en.wikipedia.org/wiki/Retina#/media/File:Fundus_photograph_of_normal_right_eye.jpg.
- [Com23b] W. Commons. *I-40W-Sunflowers*. 2023. URL: <https://commons.wikimedia.org/wiki/File:I-40W-Sunflowers.jpg>.

- [Com23c] W. Commons. *Micropapillary urothelial carcinoma*. 2023. URL: https://commons.wikimedia.org/wiki/File:Micropapillary_urothelial_carcinoma,_very_high_mag.jpg.
- [CW16a] T. S. Cohen and M. Welling. *Group Equivariant Convolutional Networks*. 2016. arXiv: 1602.07576 [cs.LG].
- [CW16b] T. S. Cohen and M. Welling. *Steerable CNNs*. 2016. arXiv: 1612.08498 [cs.LG].
- [Dos+21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV].
- [Ges+19] N. Gessert, M. Nielsen, M. Shaikh, R. Werner, and A. Schlaefer. *Skin Lesion Classification Using Ensembles of Multi-Resolution EfficientNets with Meta Data*. 2019. arXiv: 1910.03910 [cs.CV].
- [He+15] K. He, X. Zhang, S. Ren, and J. Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV].
- [HG23] D. Hendrycks and K. Gimpel. *Gaussian Error Linear Units (GELUs)*. 2023. arXiv: 1606.08415 [cs.LG].
- [How+17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. 2017. arXiv: 1704.04861 [cs.CV].
- [Hu+19] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu. *Squeeze-and-Excitation Networks*. 2019. arXiv: 1709.01507 [cs.CV].
- [Kri09] A. Krizhevsky. "Learning Multiple Layers of Features from Tiny Images." In: 2009.
- [KSH17] A. Krizhevsky, I. Sutskever, and G. E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks." In: *Commun. ACM* 60.6 (May 2017), pp. 8490. ISSN: 0001-0782. DOI: 10.1145/3065386.
- [Lar+07] H. Larochelle, D. Erhan, A. Courville, J. Bergstra, and Y. Bengio. "An Empirical Evaluation of Deep Architectures on Problems with Many Factors of Variation." In: *Proceedings of the 24th International Conference on Machine Learning*. ICML '07. Corvalis, Oregon, USA: Association for Computing Machinery, 2007, pp. 473480. ISBN: 9781595937933. DOI: 10.1145/1273496.1273556.

- [LB18] H. W. Leung and J. Bovy. “Deep learning of multi-element abundances from high-resolution spectroscopic data.” In: *Monthly Notices of the Royal Astronomical Society* (Nov. 2018). issn: 1365-2966. doi: 10.1093/mnras/sty3217.
- [Lec+98] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-based learning applied to document recognition.” In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. doi: 10.1109/5.726791.
- [Li+20] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, M. Hardt, B. Recht, and A. Talwalkar. *A System for Massively Parallel Hyperparameter Tuning*. 2020. arXiv: 1810.05934 [cs.LG].
- [Lia+18] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica. *Tune: A Research Platform for Distributed Model Selection and Training*. 2018. arXiv: 1807.05118 [cs.LG].
- [Liu+18] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, and K. Murphy. *Progressive Neural Architecture Search*. 2018. arXiv: 1712.00559 [cs.CV].
- [Liu+22] R. Liu, X. Wang, Q. Wu, L. Dai, X. Fang, T. Yan, J. Son, S. Tang, J. Li, Z. Gao, A. Galdran, J. Poorneshwaran, H. Liu, J. Wang, Y. Chen, P. Porwal, G. S. Wei Tan, X. Yang, C. Dai, H. Song, M. Chen, H. Li, W. Jia, D. Shen, B. Sheng, and P. Zhang. “DeepDRiD: Diabetic Retinopathy Grading and Image Quality Estimation Challenge.” In: *Patterns* (2022), p. 100512. issn: 2666-3899. doi: <https://doi.org/10.1016/j.patter.2022.100512>.
- [Moh+20] M. Mohamed, G. Cesa, T. S. Cohen, and M. Welling. *A Data and Compute Efficient Design for Limited-Resources Deep Learning*. 2020. arXiv: 2004.09691 [cs.LG].
- [Rau+20] J. Rauber, R. Zimmermann, M. Bethge, and W. Brendel. “Foolbox Native: Fast adversarial attacks to benchmark the robustness of machine learning models in PyTorch, TensorFlow, and JAX.” In: *Journal of Open Source Software* 5.53 (2020), p. 2607. doi: 10.21105/joss.02607.
- [RBB17] J. Rauber, W. Brendel, and M. Bethge. “Foolbox: A Python toolbox to benchmark the robustness of machine learning models.” In: *Reliable Machine Learning in the Wild Workshop, 34th International Conference on Machine Learning*. 2017.
- [Rea+19] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le. *Regularized Evolution for Image Classifier Architecture Search*. 2019. arXiv: 1802.01548 [cs.NE].

- [San+19] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. 2019. arXiv: 1801.04381 [cs.CV].
- [SH21] J. Steppan and S. Hanke. *Analysis of skin lesion images with deep learning*. 2021. arXiv: 2101.03814 [eess.IV].
- [Sze+15] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. *Rethinking the Inception Architecture for Computer Vision*. 2015. arXiv: 1512.00567 [cs.CV].
- [Tan+19] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le. *MnasNet: Platform-Aware Neural Architecture Search for Mobile*. 2019. arXiv: 1807.11626 [cs.CV].
- [TL20] M. Tan and Q. V. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. 2020. arXiv: 1905.11946 [cs.LG].
- [TRK18] P. Tschandl, C. Rosendahl, and H. Kittler. “The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions.” In: *Scientific Data* 5.1 (Aug. 2018). doi: 10.1038/sdata.2018.161.
- [Tum+23] S. Tummala, V. S. G. Thadikemalla, S. Kadry, M. Sharaf, and H. T. Rauf. “EfficientNetV2 Based Ensemble Model for Quality Estimation of Diabetic Retinopathy Images from DeepDRiD.” In: *Diagnostics* 13.4 (2023). issn: 2075-4418. doi: 10.3390/diagnostics13040622.
- [WC19] M. Weiler and G. Cesa. “General E(2)-Equivariant Steerable CNNs.” In: *Conference on Neural Information Processing Systems (NeurIPS)*. 2019.
- [Wei+18] M. Weiler, M. Geiger, M. Welling, W. Boomsma, and T. Cohen. *3D Steerable CNNs: Learning Rotationally Equivariant Features in Volumetric Data*. 2018. arXiv: 1807.02547 [cs.LG].
- [WHS18] M. Weiler, F. A. Hamprecht, and M. Storath. *Learning Steerable Filters for Rotation Equivariant CNNs*. 2018. arXiv: 1711.07289 [cs.LG].
- [WW19] D. Worrall and M. Welling. “Deep Scale-spaces: Equivariance Over Scale.” In: *Advances in Neural Information Processing Systems*. Ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett. Vol. 32. Curran Associates, Inc., 2019.
- [Yan+23] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni. “MedM-NIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification.” In: *Scientific Data* 10.1 (Jan. 2023). issn: 2052-4463. doi: 10.1038/s41597-022-01721-8.

- [ZH18] Z. Zheng and P. Hong. “Robust Detection of Adversarial Attacks by Modeling the Intrinsic Properties of Deep Neural Networks.” In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett. Vol. 31. Curran Associates, Inc., 2018.
- [Zha+23] Y. Zhang, Y. Li, Y. Li, and Z. Guo. *A Review of Adversarial Attacks in Computer Vision*. 2023. arXiv: 2308.07673 [cs.CV].
- [ZK17] S. Zagoruyko and N. Komodakis. *Wide Residual Networks*. 2017. arXiv: 1605.07146 [cs.CV].
- [ZL17] B. Zoph and Q. V. Le. *Neural Architecture Search with Reinforcement Learning*. 2017. arXiv: 1611.01578 [cs.LG].
- [Zop+18] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le. *Learning Transferable Architectures for Scalable Image Recognition*. 2018. arXiv: 1707.07012 [cs.CV].